UTS

PEMBELAJARAN MESIN



NAMA : Muhammad Hafizh Wijdan

NIM : 202431005

KELAS : H

DOSEN : Rizqia Cahyaningtyas, ST, M.Kom.

NO.PC : 08

ASISTEN : 1. MUHAMMAD ICHWAN FAUZI

2. BENEDICT SAMUEL ARDY

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

DAFTAR ISI 2
BAB I 3
PENDAHULUAN 3
BAB II 5
LANDASAN TEORI 5
BAB III 8
HASIL 8
BAB IV 37
PENUTUP 37
DAFTAR PUSTAKA 38

BAB I
PENDAHULUAN

1.1 Rumusan Masalah

Berdasarkan instruksi pengerjaan Proyek UTS Pembelajaran Mesin dan karakteristik dataset medis diabetes (02_diabetes.csv), permasalahan yang dikaji dalam praktikum ini dirumuskan sebagai berikut:

Eksplorasi dan Karakteristik Data (*Exploratory Data Analysis*): Bagaimana karakteristik distribusi statistik, pola sebaran data, serta korelasi antar atribut klinis dan demografis (gender, age, hypertension, heart_disease, smoking_history, bmi, HbA1c_level, blood_glucose_level, dan diabetes) pada dataset diabetes yang berjumlah 100.000 rekaman data?
Tahap Pra-pemrosesan Data (*Data Preprocessing*): Bagaimana tahapan *preprocessing* data yang tepat—meliputi penanganan duplikasi data, *encoding* fitur kategorikal (gender dan smoking_history), pembersihan *outlier*, serta normalisasi/standarisasi skala fitur numerik—agar perhitungan jarak (*distance metric*) seperti *Euclidean Distance* pada algoritma *K-Nearest Neighbor* (KNN) tidak mengalami bias akibat perbedaan rentang nilai antar atribut?
Implementasi Pemodelan *K-Nearest Neighbor* (KNN) Regresi: Bagaimana mengimplementasikan algoritma *K-Nearest Neighbor* (KNN) untuk tugas regresi dalam memprediksi nilai kontinu pada dataset diabetes (seperti estimasi skor risiko diabetes, kadar *blood glucose level*, atau *HbA1c level*) serta bagaimana pengaruh variasi penentuan nilai parameter k (jumlah tetangga terdekat) terhadap performa model?
Evaluasi Performa dan Akurasi Model: Seberapa baik tingkat akurasi dan performa model KNN Regresi yang dihasilkan ketika dievaluasi menggunakan metrik evaluasi regresi seperti *R-squared* (R^2 / Koefisien Determinasi), *Mean Absolute Error* (MAE), *Mean Squared Error* (MSE), dan *Root Mean Squared Error* (RMSE)?

1.2 Tujuan Masalah

Mengacu pada rumusan masalah dan tahapan pengolahan data yang telah dilakukan pada UTS praktikum, tujuan serta hasil yang ingin dicapai adalah:

Memahami Struktur dan Pola Dataset: Memperoleh pemahaman menyeluruh mengenai karakteristik dataset 02_diabetes.csv melalui proses *Exploratory Data Analysis* (EDA) dan visualisasi data untuk mengidentifikasi fitur-fitur yang memiliki pengaruh paling signifikan.
Menghasilkan Data Bersih dan Terstandarisasi: Memperoleh dataset yang telah melalui tahap *preprocessing* (*encoding* variabel kategorikal dan *feature scaling* menggunakan *StandardScaler* atau *MinMaxScaler*) sehingga siap dilatih menggunakan algoritma berbasis jarak (*distance-based algorithm*).
Membangun dan Melatih Model KNN Regresi Optimal: Menerapkan konsep *Supervised Learning* dengan algoritma *K-Nearest Neighbor* (KNN) Regresi pada data latih (*training*

data) dan menemukan konfigurasi *hyperparameter* nilai k serta metrik perhitungan jarak (*Euclidean/Manhattan/Minkowski*) yang paling optimal.
Mengukur Akurasi dan Kinerja Prediksi Model: Memperoleh nilai akurasi/kecocokan model (R^2 *Score*) dan tingkat kesalahan prediksi (MAE, MSE, RMSE) pada data uji (*testing data*) sebagai dasar analisis pengolahan data secara teoritis dan praktis.

1.3 Manfaat Masalah

Pelaksanaan dan hasil dari UTS praktikum Pembelajaran Mesin ini memberikan beberapa manfaat, baik secara teoritis maupun praktis:

Manfaat Teoritis (Akademis):
Memperdalam pemahaman mahasiswa mengenai alur kerja *Machine Learning* berdasarkan metodologi **CRISP-DM** (*Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation*).
Memberikan pemahaman komprehensif mengenai cara kerja algoritma *K-Nearest Neighbor* (KNN), khususnya bagaimana perhitungan jarak ketetanggaan (*Euclidean Distance*) dan rata-rata nilai dari k tetangga terdekat bekerja pada kasus regresi dibandingkan dengan sistem *majority voting* pada klasifikasi.

Manfaat Praktis (Teknis & Aplikatif):
Meningkatkan keterampilan teknis dalam mengolah dataset berukuran besar (100.000 baris data) menggunakan bahasa pemrograman Python beserta pustaka sains data (*Pandas, NumPy, Matplotlib, Seaborn,* dan *Scikit-Learn*).
Membuktikan secara langsung dampak krusial dari proses *feature scaling* dan pemilihan nilai k terhadap waktu komputasi serta akurasi model KNN pada data numerik dan kategorikal.
Menjadi referensi awal dalam pengembangan sistem pendukung keputusan klinis (*Clinical Decision Support System*) untuk memprediksi indikator medis atau risiko penyakit diabetes secara kuantitatif berdasarkan profil kesehatan pasien.

BAB II

LANDASAN TEORI

2.1 Konsep Dasar Machine Learning dan Alur Pengolahan Data

Machine Learning adalah cabang ilmu komputer yang memungkinkan sebuah sistem mempelajari pola (pattern) dari data historis tanpa aturan logika yang diprogram secara eksplisit [1]. Pada pemrograman konvensional, aturan dan data masukan diolah untuk menghasilkan keluaran. Pada Machine Learning, prosesnya dibalik: data masukan beserta keluaran yang benar (ground truth) diberikan kepada algoritma, lalu algoritma mempelajari fungsi pemetaan di antara keduanya. Pendekatan ini disebut Supervised Learning karena proses pelatihannya diawasi oleh data berlabel (labeled data) [1], [2]. Supervised Learning terbagi menjadi dua tugas utama, yaitu klasifikasi dan regresi [3], [4]. Klasifikasi digunakan ketika variabel target bersifat kategorikal (diskrit), misalnya "ya" atau "tidak". Regresi digunakan ketika variabel target bersifat numerik kontinu, misalnya skor risiko penyakit atau kadar glukosa darah [3], [7].

Pengerjaan praktikum ini mengikuti alur kerja (pipeline) Machine Learning yang terdiri dari enam tahap berurutan [2]:

1. **Data acquisition**: pengumpulan dan pemuatan dataset yang akan dianalisis, dalam praktikum ini data rekam medis pasien diabetes (02_diabetes.csv) [2].
2. **Exploratory Data Analysis (EDA)**: eksplorasi struktur dan karakteristik data melalui statistik deskriptif, visualisasi distribusi, analisis korelasi antarfitur, serta identifikasi missing value, duplikasi, dan outlier [5].
3. **Data preprocessing**: transformasi data mentah agar sesuai dengan kebutuhan algoritma, meliputi data cleaning (penghapusan data duplikat), encoding fitur kategorikal menjadi numerik, dan feature scaling (normalisasi atau standarisasi) [5], [6].
4. **Data splitting**: pemisahan dataset menjadi training set dan testing set (train-test split) agar performa model dapat diukur pada data yang belum pernah dilihat saat pelatihan.
5. **Model training**: pelatihan model pada training set menggunakan algoritma K-Nearest Neighbor (KNN) Regressor dan Linear Regression.
6. **Model evaluation**: pengukuran kemampuan generalisasi model pada testing set menggunakan metrik regresi, yaitu MAE, MSE, RMSE, dan R^2 [2].

2.2 Exploratory Data Analysis (EDA) dan Data Preprocessing

Sesuai instruksi praktikum, data perlu melalui EDA dan preprocessing sebelum digunakan untuk melatih model [5]. EDA merupakan investigasi awal terhadap dataset untuk memahami struktur, distribusi, dan hubungan antarvariabel, baik melalui statistik deskriptif (mean, median, nilai maksimum, dan nilai minimum) maupun visualisasi data [5], [7]. Pada dataset 02_diabetes.csv yang terdiri atas 100.000 observasi, EDA menunjukkan bahwa tidak terdapat missing value, tetapi ditemukan 3.854 baris duplikat. Analisis korelasi mengungkap kekuatan hubungan tiap fitur terhadap risiko diabetes, sedangkan pemeriksaan distribusi mengidentifikasi outlier yang perlu diperhatikan sebelum pelatihan [5], [7].

Data preprocessing diperlukan karena algoritma Machine Learning tidak dapat memproses data yang tidak bersih maupun data berformat teks [5], [6]. Tahap pertama adalah data cleaning berupa penghapusan baris duplikat agar model tidak mengalami bias akibat observasi yang berulang [5]. Tahap kedua adalah encoding, yaitu konversi fitur kategorikal menjadi representasi numerik [5],

[7]. Pada dataset ini, kolom gender (Female, Male) dan smoking_history (never, current, former, dan kategori lainnya) ditransformasi menggunakan Label Encoding atau One-Hot Encoding [5], [7].

Tahap ketiga adalah feature scaling, yang bersifat krusial untuk algoritma berbasis jarak (distance-based) seperti KNN [6]. Setiap fitur memiliki rentang nilai yang berbeda. Kolom blood_glucose_level berkisar antara 80 sampai 300, sedangkan HbA1c_level hanya berkisar antara 3,5 sampai 9,0. Tanpa penyamaan skala, fitur dengan magnitudo besar akan mendominasi perhitungan jarak. Dua metode yang umum digunakan adalah StandardScaler, yang mentransformasi data menjadi berdistribusi dengan rata-rata 0 dan simpangan baku 1 melalui $z = (x - \mu) / \sigma$, serta MinMaxScaler, yang memetakan data ke rentang tertentu melalui $x_scaled = (x - x_min) / (x_max - x_min)$. Kedua metode tersebut menghasilkan bobot skala yang setara antarfitur [6].

2.3 Algoritma K-Nearest Neighbor (KNN) dan Perhitungan Jarak

K-Nearest Neighbor (KNN) adalah algoritma Supervised Learning yang bekerja berdasarkan prinsip kemiripan antardata [4], [6]. Data baru diprediksi berdasarkan kedekatannya dengan data latih yang sudah ada dalam ruang fitur (feature space) [4]. KNN dikategorikan sebagai lazy learning karena tidak membangun model atau fungsi prediksi eksplisit pada fase pelatihan. Seluruh data latih hanya disimpan di memori, dan perhitungan jarak baru dilakukan ketika terdapat data uji yang harus diprediksi [4], [8].

Prosedur KNN terdiri dari empat langkah:

1. Penentuan nilai parameter k, yaitu jumlah tetangga terdekat yang dijadikan acuan.
2. Perhitungan jarak antara data uji dan seluruh data latih. Metrik yang paling umum adalah Euclidean Distance, yaitu jarak garis lurus antara dua titik berdasarkan teorema Pythagoras: $d = \sqrt{(\Sigma (v_1(k) - v_2(k))^2)}$, dengan k = 1 sampai N [4], [6]. Metrik alternatif meliputi Manhattan (City-Block) Distance, yaitu $d = \Sigma |v_1(k) - v_2(k)|$, dan Minkowski Distance [4].
3. Pengurutan hasil perhitungan jarak secara ascending.
4. Pemilihan k data dengan jarak terkecil sebagai tetangga terdekat.

   Pada KNN Regressor, nilai prediksi tidak ditentukan melalui majority voting seperti pada klasifikasi, tetapi melalui rata-rata nilai target dari k tetangga terdekat: $\hat{y} = (1/k) \Sigma y_i$, dengan i = 1 sampai k [3], [4], [8]. Kelebihan KNN adalah konsepnya yang sederhana dan ketahanannya terhadap noisy data. Kekurangannya adalah beban komputasi yang tinggi pada fase pengujian karena jarak harus dihitung terhadap puluhan ribu observasi, serta sensitivitas yang tinggi terhadap skala fitur [4], [6].

2.4 Algoritma Linear Regression

Linear Regression adalah algoritma yang umum digunakan untuk memprediksi nilai kontinu dan berfungsi sebagai pembanding KNN Regressor. Berbeda dengan KNN yang berbasis pencarian tetangga terdekat, Linear Regression menggunakan pendekatan parametrik, yaitu membentuk garis lurus (atau hyperplane pada ruang berdimensi lebih tinggi) yang paling sesuai untuk memodelkan hubungan antara variabel independen (fitur masukan) dan variabel dependen (target keluaran).

Pada Simple Linear Regression yang hanya melibatkan satu fitur, persamaannya adalah $y = \beta_0 + \beta_1 x$. Dataset medis umumnya melibatkan banyak faktor sekaligus, seperti usia, BMI, dan tekanan darah, sehingga digunakan Multiple Linear Regression dengan persamaan:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

Dalam persamaan tersebut, y adalah nilai target yang diprediksi (misalnya kadar glukosa darah), β_0 adalah intercept, yaitu titik potong sumbu Y ketika seluruh nilai x sama dengan nol, dan β_1 sampai β_n adalah koefisien regresi yang menyatakan besarnya pengaruh tiap fitur x terhadap nilai prediksi.

Estimasi parameter umumnya dilakukan dengan metode Ordinary Least Squares (OLS), yang bertujuan meminimalkan total galat (error) antara nilai prediksi dan nilai aktual. Nilai β dicari sehingga cost function berupa Sum of Squared Residuals (SSR) mencapai minimum:

$SSR = \Sigma (y_i - \hat{y}_i)^2$, dengan i = 1 sampai n

Keunggulan Linear Regression terletak pada efisiensi komputasi saat pelatihan maupun prediksi, serta interpretabilitas (explainability) yang tinggi. Melalui nilai koefisien β, kontribusi tiap fitur dalam menaikkan atau menurunkan nilai target dapat diketahui secara langsung. Kelemahannya, algoritma ini mengasumsikan hubungan linier antarvariabel dan sensitif terhadap outlier ekstrem yang tidak ditangani pada tahap EDA.

2.5 Konteks Dataset Diabetes dan Evaluasi Performa Model

Dataset 02_diabetes.csv berisi catatan medis pasien terkait Diabetes Mellitus, yaitu penyakit metabolik kronis yang ditandai oleh kadar glukosa darah yang tinggi [1], [7]. Dataset ini memiliki sembilan atribut, yaitu gender, usia (age), hipertensi (hypertension), penyakit jantung (heart_disease), riwayat merokok (smoking_history), indeks massa tubuh (bmi), kadar HbA1c (HbA1c_level), kadar glukosa darah (blood_glucose_level), dan label diabetes [1], [5], [7]. Secara medis, HbA1c_level dan blood_glucose_level merupakan indikator utama kadar gula darah, sedangkan usia, BMI, hipertensi, dan penyakit jantung berperan sebagai faktor risiko tambahan [1], [5], [7].

Setelah dilatih dengan KNN Regressor dan Linear Regression, performa model dievaluasi pada testing set [8], [10]. Karena keduanya merupakan model regresi dengan target numerik kontinu, evaluasi dilakukan dengan membandingkan nilai prediksi ($\hat{y}_i$) terhadap nilai aktual (y_i) menggunakan empat metrik [8], [10]:

- **Mean Absolute Error (MAE)**: rata-rata selisih absolut antara nilai prediksi dan nilai aktual. $MAE = (1/n) \Sigma |y_i - \hat{y}_i|$.
- **Mean Squared Error (MSE)**: rata-rata kuadrat selisih, sehingga prediksi dengan galat besar memberikan penalti yang lebih berat. $MSE = (1/n) \Sigma (y_i - \hat{y}_i)^2$.
- **Root Mean Squared Error (RMSE)**: akar kuadrat dari MSE ($RMSE = \sqrt{MSE}$), sehingga satuannya kembali sama dengan satuan target dan lebih mudah diinterpretasikan.
- **R-Squared (R^2)**: proporsi variansi target yang dapat dijelaskan oleh model. $R^2 = 1 - \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2$ [10].

Semakin kecil nilai MAE, MSE, dan RMSE (mendekati 0) serta semakin besar nilai R^2 (mendekati 1 atau 100%), semakin akurat model regresi dalam menangkap pola pada dataset diabetes [8], [10].

BAB III

HASIL

Cell 1: Inisialisasi Pustaka dan Konfigurasi Visualisasi

Maksud dan Tujuan Cell: Mengimpor semua paket modul Python pihak ketiga yang diperlukan selama siklus machine learning, mulai dari manipulasi tabel, pembuatan plot visual, modul preprocessing scikit-learn, algoritma model (Regresi Linier dan KNN), hingga seluruh fungsi kalkulasi metrik evaluasi. Konfigurasi grid grafik juga disiapkan di sini agar seluruh visualisasi seragam.

Kode Program:

```
# Muhammad Hafizh Wijdan - 202431005

# library buat manipulasi dataframe dan array numerik
import pandas as pd
import numpy as np

# library buat bikin grafik dan visualisasi data
import matplotlib.pyplot as plt

# modul sklearn buat split data latih dan data uji
from sklearn.model_selection import train_test_split

# modul buat ubah kategori ke angka dan standarisasi nilai
from sklearn.preprocessing import StandardScaler, LabelEncoder

# algoritma regresi linier dan k-nearest neighbors
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor

# kumpulan fungsi buat hitung akurasi, confusion matrix, dan error
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report,
    mean_absolute_error, mean_squared_error, r2_score
)

# pasang tema visualisasi biar grafiknya rapi ada grid
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available
else 'default')
%matplotlib inline

print("Semua library berhasil dimuat dan siap digunakan.")
```

Penjelasan Kode Per Baris:

Baris 1: Komentar watermark tanda identitas mahasiswa (Muhammad Hafizh Wijdan - 202431005).

Baris 4-5: Mengimpor pandas untuk manipulasi struktur DataFrame dan numpy untuk komputasi matriks larik numerik.

Baris 8: Mengimpor matplotlib.pyplot untuk merancang plot dan grafik visual.

Baris 11: Mengimpor train_test_split dari sklearn untuk memisahkan dataset menjadi porsi data latih dan uji.

Baris 14: Mengimpor StandardScaler untuk penskalaan data dan LabelEncoder untuk menyandikan teks kategori menjadi angka.

Baris 17-18: Mengimpor kelas algoritma LinearRegression, KNeighborsClassifier (klasifikasi), dan KNeighborsRegressor (regresi kontinu).

Baris 21-25: Mengimpor fungsi metrik pengujian: akurasi, presisi, recall, F1, confusion matrix, classification report, MAE, MSE, dan R2.

Baris 28-29: Mengatur tema grafik seaborn-whitegrid dan mengaktifkan inline plotting di Jupyter Notebook.

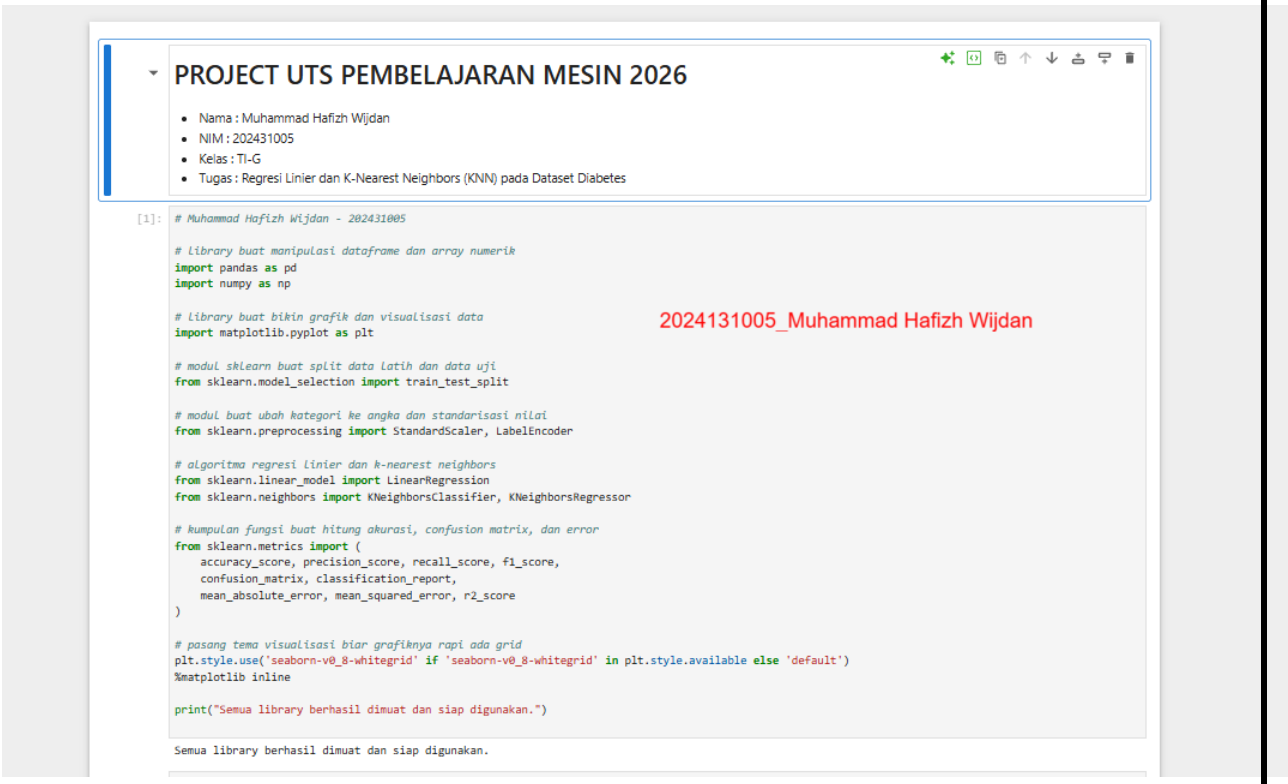
Baris 31: Mencetak teks notifikasi bahwa semua library berhasil dimuat.

Hasil Output dan Maknanya:

Hasil Output dan Maknanya:

- Teks Output: 'Semua library berhasil dimuat dan siap digunakan.'

- Makna Output: Membuktikan bahwa seluruh pustaka eksternal (pandas, numpy, matplotlib, dan scikit-learn) telah terpasang dengan benar di lingkungan Python dan siap mengeksekusi operasi matematika dan pemodelan data pada cell berikutnya tanpa ada pesan error 'ModuleNotFoundError'.



Cell 2: Pemuatan Dataset dan Penelusuran Dimensi Awal

Maksud dan Tujuan Cell: Membaca berkas data mentah 02_diabetes.csv ke memori menggunakan pandas, memeriksa jumlah baris observasi dan jumlah fitur kolom, mengintip 5 sampel data pertama untuk memahami konteks atribut medisnya, serta meninjau informasi tipe data setiap kolom guna memastikan data terbaca dengan tepat.

Kode Program:

```
# Muhammad Hafizh Wijdan - 202431005

# baca file csv dataset diabetes
df = pd.read_csv('02_diabetes.csv')
```

```
# cek berapa jumlah baris dan kolom datanya
print(f"Ukuran dataset: {df.shape[0]} baris, {df.shape[1]} kolom\n")

# lihat 5 baris pertama data buat tahu isinya
print("--- 5 Baris Teratas ---")
display(df.head())

# cek tipe data tiap kolom dan pastikan kebaca semua
print("\n--- Informasi Tipe Data ---")
df.info()
```

Penjelasan Kode Per Baris:

Baris 1: Watermark identitas mahasiswa.

Baris 4: Membaca berkas 02_diabetes.csv menggunakan fungsi pd.read_csv() dan menyimpannya ke variabel df.

Baris 7: Menampilkan total baris dan kolom dataset lewat atribut df.shape.

Baris 10-11: Menampilkan 5 rekaman data pertama menggunakan display(df.head()).

Baris 14-15: Menampilkan ringkasan tipe data setiap kolom dan informasi ketersediaan nilai non-null dengan df.info().

Hasil Output dan Maknanya:

Hasil Output dan Maknanya:

- Dimensi Data: Ukuran dataset adalah 100.000 baris dan 9 kolom.

- 5 Baris Teratas: Memperlihatkan profil pasien dengan atribut gender, age, hypertension, heart_disease, smoking_history, bmi, HbA1c_level, blood_glucose_level, dan label target diabetes (0 = tidak diabetes, 1 = diabetes).

- Info Tipe Data: Terdapat 3 kolom float64 (age, bmi, HbA1c_level), 4 kolom int64 (hypertension, heart_disease, blood_glucose_level, diabetes), dan 2 kolom bertipe teks/object (gender dan smoking_history) yang nantinya wajib dikonversi menjadi angka agar bisa diolah oleh model machine learning.

[2]:
```
# Muhammad Hafizh Wijdan - 202431005

# baca file csv dataset diabetes
df = pd.read_csv('02_diabetes.csv')

# cek berapa jumlah baris dan kolom datanya
print(f"Ukuran dataset: {df.shape[0]} baris, {df.shape[1]} kolom\n")

# lihat 5 baris pertama data buat tahu isinya
print("--- 5 Baris Teratas ---")
display(df.head())

# cek tipe data tiap kolom dan pastikan kebaca semua
print("\n--- Informasi Tipe Data ---")
df.info()
```

```
Ukuran dataset: 100000 baris, 9 kolom

--- 5 Baris Teratas ---
```

2024131005_Muhammad Hafizh Wijdan

	gender	age	hypertension	heart_disease	smoking_history	bmi	HbA1c_level	blood_glucose_level	diabetes
0	Female	80.0	0	1	never	25.19	6.6	140	0
1	Female	54.0	0	0	No Info	27.32	6.6	80	0
2	Male	28.0	0	0	never	27.32	5.7	158	0
3	Female	36.0	0	0	current	23.45	5.0	155	0
4	Male	76.0	1	1	current	20.14	4.8	155	0

```
--- Informasi Tipe Data ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 9 columns):
 #   Column               Non-Null Count   Dtype
---  ------               --------------   -----
 0   gender               100000 non-null  object
 1   age                  100000 non-null  float64
 2   hypertension         100000 non-null  int64
 3   heart_disease        100000 non-null  int64
 4   smoking_history      100000 non-null  object
 5   bmi                  100000 non-null  float64
 6   HbA1c_level          100000 non-null  float64
 7   blood_glucose_level  100000 non-null  int64
 8   diabetes             100000 non-null  int64
dtypes: float64(3), int64(4), object(2)
memory usage: 6.9+ MB
```

Cell 3: Pemeriksaan Kualitas Data dan Statistik Deskriptif

Maksud dan Tujuan Cell: Melakukan audit kebersihan data (data sanity check) dengan mendeteksi adanya sel yang kosong atau hilang (missing values), menghitung jumlah baris data yang kembar identik (duplikat), serta menghitung ringkasan statistik deskriptif untuk melihat rata-rata, rentang nilai minimum, maksimum, dan sebaran kuartil tiap variabel medis numerik.

Kode Program:

```
# Muhammad Hafizh Wijdan - 202431005

# hitung apakah ada nilai kosong atau null di setiap kolom
print("--- Cek Missing Values ---")
print(df.isnull().sum())

# hitung ada berapa baris data yang duplikat
print(f"\nJumlah data duplikat: {df.duplicated().sum()} baris ({df.duplicated().sum() /
len(df)  100:.2f}%)")

# lihat statistik deskriptif buat tahu rata-rata, min, dan max tiap fitur
print("\n--- Ringkasan Statistik Data Numerik ---")
display(df.describe())
```

Penjelasan Kode Per Baris:

Baris 1: Watermark identitas mahasiswa.

Baris 4-5: Menghitung total nilai kosong pada tiap kolom memakai df.isnull().sum().

Baris 8: Menghitung total baris duplikat identik menggunakan df.duplicated().sum() dan persentasenya terhadap total baris.

Baris 11-12: Menampilkan tabel ringkasan ukuran pemusatan dan penyebaran statistik angka dengan display(df.describe()).

Hasil Output dan Maknanya:

Hasil Output dan Maknanya:

- Missing Values: Seluruh kolom menghasilkan nilai 0, membuktikan dataset tidak memiliki data bolong/hilang.

- Data Duplikat: Ditemukan 3.854 baris data kembar (3,85%). Data duplikat ini berbahaya jika dibiarkan karena bisa memicu data leakage (kebocoran data antara set latih dan uji) sehingga wajib dibersihkan pada tahap berikutnya.

- Statistik Deskriptif: Rata-rata usia pasien adalah 41,8 tahun (rentang 0,08 sampai 80 tahun). Rata-rata glukosa darah adalah 138 mg/dL dengan batas maksimum 300 mg/dL, dan kadar HbA1c rata-rata 5,5% dengan batas tertinggi 9,0%.

[3]:
```
# Muhammad Hafizh Wijdan - 202431005

# hitung apakah ada nilai kosong atau null di setiap kolom
print("--- Cek Missing Values ---")
print(df.isnull().sum())

# hitung ada berapa baris data yang duplikat
print(f"\nJumlah data duplikat: {df.duplicated().sum()} baris ({df.duplicated().sum() / len(df) * 100:.2f}%)")

# lihat statistik deskriptif buat tahu rata-rata, min, dan max tiap fitur
print("\n--- Ringkasan Statistik Data Numerik ---")
display(df.describe())
```

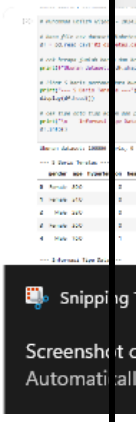
```
--- Cek Missing Values ---
gender                0
age                   0
hypertension          0
heart_disease         0
smoking_history       0
bmi                   0
HbA1c_level           0
blood_glucose_level   0
diabetes              0
dtype: int64

Jumlah data duplikat: 3854 baris (3.85%)

--- Ringkasan Statistik Data Numerik ---
```

2024131005_Muhammad Hafizh Wijdan

	age	hypertension	heart_disease	bmi	HbA1c_level	blood_glucose_level	diabetes
count	100000.000000	100000.00000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000
mean	41.885856	0.07485	0.039420	27.320767	5.527507	138.058060	0.085000
std	22.516840	0.26315	0.194593	6.636783	1.070672	40.708136	0.278883
min	0.080000	0.00000	0.000000	10.010000	3.500000	80.000000	0.000000
25%	24.000000	0.00000	0.000000	23.630000	4.800000	100.000000	0.000000
50%	43.000000	0.00000	0.000000	27.320000	5.800000	140.000000	0.000000
75%	60.000000	0.00000	0.000000	29.580000	6.200000	159.000000	0.000000
max	80.000000	1.00000	1.000000	95.690000	9.000000	300.000000	1.000000



Cell 4: Analisis Visual Sebaran Target Diabetes (Class Imbalance)

Maksud dan Tujuan Cell: Menggambarkan visualisasi diagram batang dan diagram lingkaran (pie chart) untuk menganalisis rasio perbandingan kelas target diabetes. Langkah ini bertujuan untuk membuktikan secara empiris ada tidaknya permasalahan ketidakseimbangan kelas (class imbalance problem) yang sangat lazim terjadi pada data rekam medis.

Kode Program:

```
# Muhammad Hafizh Wijdan - 202431005

# siapkan area canvas dengan 2 kolom grafik
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# hitung jumlah masing-masing kelas target diabetes (0 dan 1)
counts = df['diabetes'].value_counts()
```

```
labels = ['Non-Diabetes (0)', 'Diabetes (1)']
colors = ['#2b5c8f', '#d9534f']

# buat diagram batang di sebelah kiri
axes[0].bar(labels, counts.values, color=colors, edgecolor='black', width=0.5)
axes[0].set_title('Jumlah Pasien Berdasarkan Status Diabetes', fontsize=12,
fontweight='bold')
axes[0].set_ylabel('Jumlah Orang')
axes[0].grid(axis='y', linestyle='--', alpha=0.7)

# pasang angka persentase di atas batang grafik
for i, v in enumerate(counts.values):
    axes[0].text(i, v + 1200, f"{v:,} ({v/len(df)100:.1f}%)", ha='center',
fontweight='bold')

# buat diagram lingkaran (pie chart) di sebelah kanan
axes[1].pie(counts.values, labels=labels, autopct='%1.1f%%', startangle=140,
            colors=colors, explode=(0, 0.1), shadow=True,
            textprops={'fontsize': 10, 'fontweight': 'bold'})
axes[1].set_title('Persentase Target Diabetes (Class Imbalance)', fontsize=12,
fontweight='bold')

# watermark nim pada grafik
plt.figtext(0.99, 0.015, 'NIM: 202431005 - Muhammad Hafizh Wijdan', ha='right',
va='bottom', fontsize=9, color='#FF0000', fontstyle='italic', weight='bold')

plt.tight_layout()
plt.show()

print(f"Hasil distribusi target: {counts[0]:,} orang sehat (0) dan {counts[1]:,} orang
diabetes (1)")
```

Penjelasan Kode Per Baris:

Baris 1: Watermark identitas mahasiswa.

Baris 4: Menyiapkan area kanvas 1 baris x 2 kolom subplot berukuran 12x5 inci.

Baris 7-9: Menghitung frekuensi target dengan value_counts(), menyiapkan label teks, dan warna batang diagram.

Baris 12-15: Menggambar diagram batang di subplot kiri lengkap dengan judul, label Y, dan garis kisi horizontal.

Baris 18-19: Melakukan iterasi untuk menuliskan angka jumlah orang dan persentasenya tepat di atas setiap balok diagram batang.

Baris 22-25: Menggambar diagram lingkaran di subplot kanan dengan potongan diabetes diberi efek pecahan (explode 0.1).

Baris 28: Menyematkan watermark identitas mahasiswa pada posisi pojok bawah grafik.

Baris 30-31: Merapikan tata letak gambar dengan tight_layout() dan memunculkan grafik ke layar.

Baris 33: Mencetak ringkasan distribusi target ke konsol.

Hasil Output dan Maknanya:

Hasil Output dan Maknanya:

- Teks Output: 'Hasil distribusi target: 91,500 orang sehat (0) dan 8,500 orang diabetes (1)'.

- Visualisasi Grafik: Diagram batang dan pie chart menunjukkan proporsi 91,5% pasien Non-Diabetes (0) berbanding 8,5% pasien Diabetes (1).

- Makna Klinis/Analitis: Terjadi ketidakseimbangan kelas yang cukup ekstrem (lebih dari 10:1). Implikasinya, metrik evaluasi model tidak boleh hanya berpatokan pada akurasi semata, melainkan wajib mengecek nilai Presisi, Recall (Sensitivitas), dan F1-Score agar model tidak bias ke kelas mayoritas pasien sehat.

[4]:
```
# Muhammad Hafizh Wijdan - 202431005

# siapkan area canvas dengan 2 kolom grafik
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# hitung jumlah masing-masing kelas target diabetes (0 dan 1)
counts = df['diabetes'].value_counts()
labels = ['Non-Diabetes (0)', 'Diabetes (1)']
colors = ['#2b5c8f', '#d9534f']

# buat diagram batang di sebelah kiri
axes[0].bar(labels, counts.values, color=colors, edgecolor='black', width=0.5)
axes[0].set_title('Jumlah Pasien Berdasarkan Status Diabetes', fontsize=12, fontweight='bold')
axes[0].set_ylabel('Jumlah Orang')
axes[0].grid(axis='y', linestyle='--', alpha=0.7)

# pasang angka persentase di atas batang grafik
for i, v in enumerate(counts.values):
    axes[0].text(i, v + 1200, f"{v:,} ({v/len(df)*100:.1f}%)", ha='center', fontweight='bold')

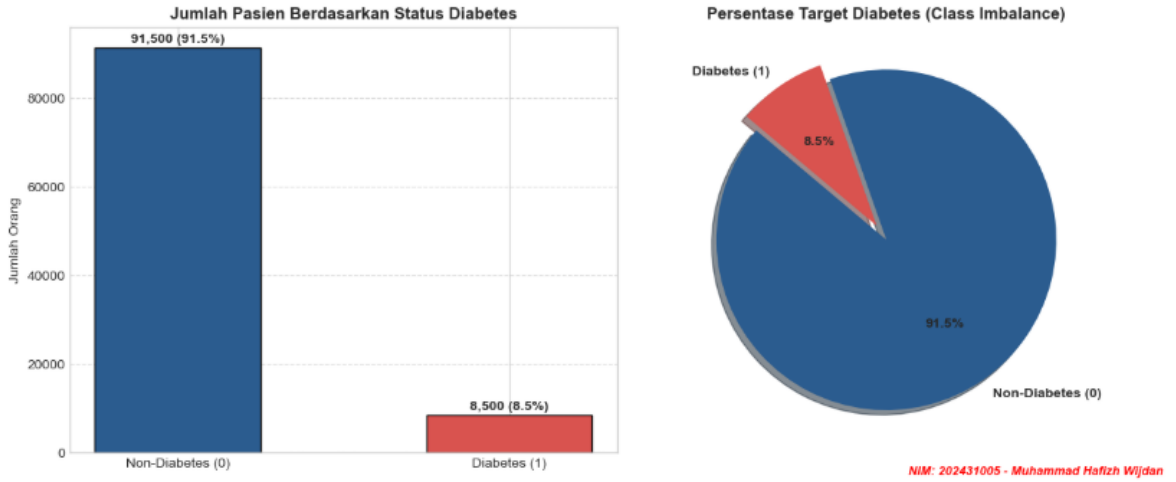
# buat diagram lingkaran (pie chart) di sebelah kanan
axes[1].pie(counts.values, labels=labels, autopct='%1.1f%%', startangle=140,
            colors=colors, explode=(0, 0.1), shadow=True,
            textprops={'fontsize': 10, 'fontweight': 'bold'})
axes[1].set_title('Persentase Target Diabetes (Class Imbalance)', fontsize=12, fontweight='bold')

# watermark nim pada grafik
plt.figtext(0.99, 0.015, 'NIM: 202431005 - Muhammad Hafizh Wijdan', ha='right', va='bottom', fontsize=9, color='#FF0000', fontstyle='italic', weight='b

plt.tight_layout()
plt.show()

print(f"Hasil distribusi target: {counts[0]:,} orang sehat (0) dan {counts[1]:,} orang diabetes (1)")
```

202431005_Muhammad Hafizh Wijdan




```
Hasil distribusi target: 91,500 orang sehat (0) dan 8,500 orang diabetes (1)
```

Cell 5: Analisis Distribusi Variabel Medis Numerik

Maksud dan Tujuan Cell: Membuat empat diagram histogram dalam kisi 2x2 untuk meneliti bentuk kurva sebaran data pada empat parameter klinis utama: usia pasien (age), indeks massa tubuh (bmi), kadar hemoglobin terglikasi (HbA1c_level), dan kadar gula darah puasa (blood_glucose_level).

Kode Program:

```
# Muhammad Hafizh Wijdan - 202431005

# siapkan canvas 2 baris x 2 kolom buat tampilin 4 histogram
fig, axes = plt.subplots(2, 2, figsize=(13, 8))

# daftar nama kolom dan judul grafik masing-masing
features_plot = ['age', 'bmi', 'HbA1c_level', 'blood_glucose_level']
```

```
titles = ['Distribusi Usia Pasien', 'Distribusi BMI Pasien',
          'Distribusi Kadar HbA1c', 'Distribusi Kadar Glukosa Darah']
colors_hist = ['#1f77b4', '#2ca02c', '#ff7f0e', '#d62728']

# looping buat gambar histogram di setiap subplot
for ax, feat, title, col in zip(axes.flatten(), features_plot, titles, colors_hist):
    ax.hist(df[feat], bins=30, color=col, edgecolor='black', alpha=0.75)
    ax.set_title(title, fontsize=11, fontweight='bold')
    ax.set_xlabel(feat)
    ax.set_ylabel('Jumlah Pasien')
    ax.grid(axis='y', linestyle='--', alpha=0.6)

# watermark nim pada grafik
plt.figtext(0.99, 0.015, 'NIM: 202431005 - Muhammad Hafizh Wijdan', ha='right',
va='bottom', fontsize=9, color='#FF0000', fontstyle='italic', weight='bold')

plt.tight_layout()
plt.show()
```

Penjelasan Kode Per Baris:

Baris 1: Watermark identitas mahasiswa.

Baris 4: Menyiapkan kanvas 2 baris x 2 kolom subplot berukuran 13x8 inci.

Baris 7-10: Menentukan daftar nama kolom, judul plot, dan variasi warna masing-masing histogram.

Baris 13-18: Melakukan looping untuk menggambar histogram ax.hist() dengan pembagian 30 bin pada setiap subplot.

Baris 21: Menyematkan watermark identitas mahasiswa pada grafik.

Baris 23-24: Menyesuaikan tata letak gambar dan menampilkan visualisasi histogram ke layar.

Hasil Output dan Maknanya:

Hasil Output dan Maknanya:

- Visualisasi Grafik: Terbentuk 4 histogram berwarna (biru untuk usia, hijau untuk BMI, oranye untuk HbA1c, dan merah untuk glukosa darah).

- Makna Sebaran:

1. Usia: Memiliki sebaran merata dari anak-anak hingga lansia dengan puncak frekuensi pada usia 50-80 tahun.

2. BMI: Terdistribusi mirip lonceng normal yang berpusat di rentang 25-30 (kategori overweight), dengan ekor kanan panjang hingga 95 (obesitas ekstrem).

3. HbA1c: Terpusat di rentang nilai 5.5 - 6.5%, yang merupakan batas transisi antara pradiabetes dan diabetes.

4. Glukosa Darah: Memiliki konsentrasi frekuensi tinggi pada rentang normal (80-160 mg/dL), dan sejumlah pasien menyebar di atas 200 mg/dL (hiperglikemia).

```
[5]: # Muhammad Hafizh Wijdan - 202431005

     # siapkan canvas 2 baris x 2 kolom buat tampilin 4 histogram
     fig, axes = plt.subplots(2, 2, figsize=(13, 8))

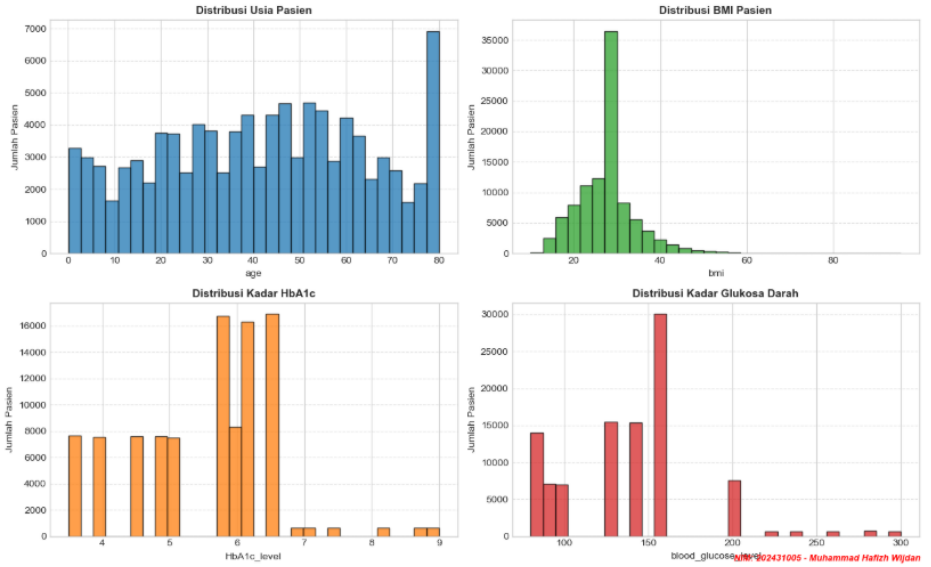
     # daftar nama kolom dan judul grafik masing-masing
     features_plot = ['age', 'bmi', 'HbA1c_level', 'blood_glucose_level']
     titles = ['Distribusi Usia Pasien', 'Distribusi BMI Pasien',
               'Distribusi Kadar HbA1c', 'Distribusi Kadar Glukosa Darah']
     colors_hist = ['#1f77b4', '#2ca02c', '#ff7f0e', '#d62728']

     # Looping buat gambar histogram di setiap subplot
     for ax, feat, title, col in zip(axes.flatten(), features_plot, titles, colors_hist):
         ax.hist(df[feat], bins=30, color=col, edgecolor='black', alpha=0.75)
         ax.set_title(title, fontsize=11, fontweight='bold')
         ax.set_xlabel(feat)
         ax.set_ylabel('Jumlah Pasien')
         ax.grid(axis='y', linestyle='--', alpha=0.6)

     # watermark nim pada grafik
     plt.figtext(0.99, 0.015, 'NIM: 202431005 - Muhammad Hafizh Wijdan', ha='right', va='bottom', fontsize=9, color='#FF0000', fontstyle='italic', weight='b

     plt.tight_layout()
     plt.show()
```

2024131005_Muhammad Hafizh Wijdan



Cell 6: Analisis Matriks Korelasi Linear Fitur

Maksud dan Tujuan Cell: Menghitung derajat keeratan hubungan linear antar variabel numerik menggunakan koefisien korelasi Pearson dan menyajikannya dalam bentuk peta panas warna (heatmap) matriks korelasi. Tujuannya adalah mendeteksi secara kuantitatif faktor risiko mana yang paling berkontribusi terhadap diabetes serta mengecek multikolinearitas antar prediktor.

Kode Program:

```
# Muhammad Hafizh Wijdan - 202431005

# pilih fitur numerik buat dihitung nilai korelasinya
num_cols = ['age', 'hypertension', 'heart_disease', 'bmi', 'HbA1c_level',
'blood_glucose_level', 'diabetes']
corr_matrix = df[num_cols].corr()

# bikin canvas heatmap korelasi
fig, ax = plt.subplots(figsize=(8, 6.5))
cax = ax.matshow(corr_matrix, cmap='coolwarm', vmin=-1, vmax=1)
fig.colorbar(cax)

# pasang nama kolom pada sumbu x dan y
ax.set_xticks(range(len(num_cols)))
ax.set_yticks(range(len(num_cols)))
ax.set_xticklabels(num_cols, rotation=45, ha='left', fontsize=9)
ax.set_yticklabels(num_cols, fontsize=9)

# tuliskan angka korelasi di setiap kotak heatmap
```

```
for i in range(len(num_cols)):
    for j in range(len(num_cols)):
        val = corr_matrix.iloc[i, j]
        ax.text(j, i, f"{val:.2f}", ha='center', va='center',
                color='white' if abs(val) > 0.4 else 'black', fontweight='bold',
fontsize=9)

ax.set_title('Korelasi Antar Fitur Medis terhadap Status Diabetes', fontsize=12,
fontweight='bold', pad=25)

# watermark nim pada grafik
plt.figtext(0.99, 0.015, 'NIM: 202431005 - Muhammad Hafizh Wijdan', ha='right',
va='bottom', fontsize=9, color='#FF0000', fontstyle='italic', weight='bold')

plt.tight_layout()
plt.show()

# cetak urutan korelasi tertinggi ke terendah terhadap diabetes
print("--- Urutan Korelasi Fitur terhadap Diabetes ---")
print(corr_matrix['diabetes'].sort_values(ascending=False))
```

Penjelasan Kode Per Baris:

Baris 1: Watermark identitas mahasiswa.

Baris 4-5: Memilih fitur numerik dan menghitung matriks korelasi Pearson menggunakan df[num_cols].corr().

Baris 8-10: Menyiapkan kanvas heatmap, memetakan gradasi warna coolwarm (-1 s.d. +1), dan memunculkan colorbar.

Baris 13-16: Mengatur posisi sumbu dan memasang nama fitur pada sumbu X (dimiringkan 45 derajat) dan sumbu Y.

Baris 19-23: Melakukan perulangan untuk menuliskan angka nilai korelasi dua desimal di setiap kotak sel heatmap.

Baris 25: Memasang judul grafik korelasi faktor risiko terhadap status diabetes.

Baris 28: Menyematkan watermark identitas mahasiswa pada grafik.

Baris 30-31: Merapikan tata letak dan memunculkan gambar heatmap.

Baris 34-35: Mencetak urutan faktor risiko dengan korelasi tertinggi terhadap diabetes.

Hasil Output dan Maknanya:

Hasil Output dan Maknanya:

- Visualisasi Heatmap: Matriks berwarna dengan angka korelasi di setiap sel (-1 hingga +1).

- Urutan Korelasi terhadap Diabetes:

1. blood_glucose_level = +0.42 (korelasi positif paling kuat)
2. HbA1c_level = +0.40 (korelasi positif kuat kedua)
3. age = +0.26 (lansia cenderung lebih berisiko)
4. bmi = +0.21 (kelebihan berat badan meningkatkan risiko)
5. hypertension = +0.20 dan heart_disease = +0.17

- Makna Analitis: Parameter biokimia darah (glukosa dan HbA1c) merupakan prediktor paling signifikan dalam menentukan apakah seorang pasien mengidap diabetes atau tidak.

[6]:
```
# Muhammad Hafizh Wijdan - 202431005

# pilih fitur numerik buat dihitung nilai korelasinya
num_cols = ['age', 'hypertension', 'heart_disease', 'bmi', 'HbA1c_level', 'blood_glucose_level', 'diabetes']
corr_matrix = df[num_cols].corr()

# bikin canvas heatmap korelasi
fig, ax = plt.subplots(figsize=(8, 6.5))
cax = ax.matshow(corr_matrix, cmap='coolwarm', vmin=-1, vmax=1)
fig.colorbar(cax)

# pasang nama kolom pada sumbu x dan y
ax.set_xticks(range(len(num_cols)))
ax.set_yticks(range(len(num_cols)))
ax.set_xticklabels(num_cols, rotation=45, ha='left', fontsize=9)
ax.set_yticklabels(num_cols, fontsize=9)

# tuliskan angka korelasi di setiap kotak heatmap
for i in range(len(num_cols)):
    for j in range(len(num_cols)):
        val = corr_matrix.iloc[i, j]
        ax.text(j, i, f"{val:.2f}", ha='center', va='center',
                color='white' if abs(val) > 0.4 else 'black', fontweight='bold', fontsize=9)

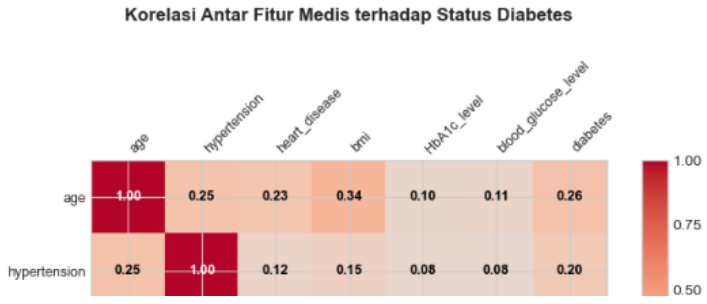
ax.set_title('Korelasi Antar Fitur Medis terhadap Status Diabetes', fontsize=12, fontweight='bold', pad=25)

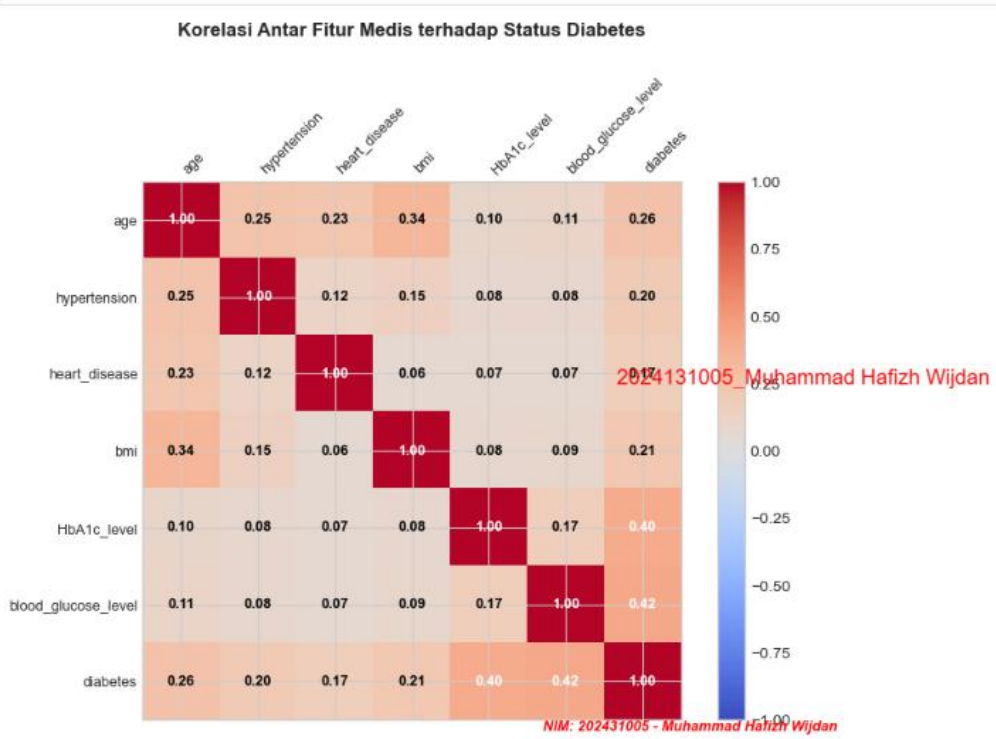
# watermark nim pada grafik
plt.figtext(0.99, 0.015, 'NIM: 202431005 - Muhammad Hafizh Wijdan', ha='right', va='bottom', fontsize=9, color='#FF0000', fontstyle='italic', weight='

plt.tight_layout()
plt.show()

# cetak urutan korelasi tertinggi ke terendah terhadap diabetes
print("--- Urutan Korelasi Fitur terhadap Diabetes ---")
print(corr_matrix['diabetes'].sort_values(ascending=False))
```

2024131005_Muhammad Hafizh Wijdan






```
--- Urutan Korelasi Fitur terhadap Diabetes ---
diabetes              1.000000
blood_glucose_level   0.419558
HbA1c_level           0.400660
age                   0.258008
bmi                   0.214357
hypertension          0.197823
heart_disease         0.171727
Name: diabetes, dtype: float64
```

Cell 7: Pembersihan Data Duplikat, Anomali, dan Encoding Fitur

Maksud dan Tujuan Cell: Menjalankan tahap data preprocessing inti: membuang 3.854 baris duplikat agar data uji bersih dari duplikasi data latih, membuang 18 baris anomali pada kolom gender ('Other'), mengubah data teks biner gender menjadi angka (0/1) dengan LabelEncoder, serta mengubah kolom teks kategori riwayat merokok (smoking_history) menjadi kolom angka biner baru memakai One-Hot Encoding (pd.get_dummies).

Kode Program:

```
# Muhammad Hafizh Wijdan - 202431005

# 1. bersihkan data duplikat biar gak bocor pas testing
total_awal = len(df)
df_clean = df.drop_duplicates().copy()
print(f"Data duplikat yang dibuang: {total_awal - len(df_clean)} baris")

# 2. buang baris dengan gender 'Other' karena cuma 18 baris
df_clean = df_clean[df_clean['gender'] != 'Other'].copy()
print(f"Sisa data setelah filter anomali: {df_clean.shape[0]} baris")

# 3. ubah jenis kelamin jadi angka: Female=0, Male=1
le_gender = LabelEncoder()
df_clean['gender'] = le_gender.fit_transform(df_clean['gender'])

# 4. ubah kolom riwayat merokok jadi one-hot encoding
df_encoded = pd.get_dummies(df_clean, columns=['smoking_history'], drop_first=True)

print(f"Bentuk data setelah encoding: {df_encoded.shape[0]} baris dan
{df_encoded.shape[1]} kolom")
```

```
print("\n--- 5 Baris Data Bersih ---")
display(df_encoded.head())
```

Penjelasan Kode Per Baris:

Baris 1: Watermark identitas mahasiswa.

Baris 4-6: Menghapus data kembar memakai drop_duplicates() dan mencetak jumlah baris yang dibuang (3.854 baris).

Baris 9-10: Memfilter kolom gender agar hanya menyisakan Female dan Male (membuang 18 baris anomali).

Baris 13-14: Mengubah teks gender menjadi biner angka (0 = Female, 1 = Male) menggunakan LabelEncoder.

Baris 17: Menerapkan One-Hot Encoding pada riwayat merokok menggunakan pd.get_dummies(drop_first=True).

Baris 19-21: Menampilkan dimensi dataset bersih (96.128 baris x 13 kolom) dan menampilkan 5 baris pertamanya.

Hasil Output dan Maknanya:

Hasil Output dan Maknanya:

- Data Duplikat Dibuang: 3.854 baris.

- Sisa Data Bersih: 96.128 baris (dari total awal 100.000 baris).

- Bentuk Data Baru: 96.128 baris dan 13 kolom numerik siap latih.

- Makna Preprocessing: Seluruh kolom teks telah berhasil ditransformasikan menjadi matriks angka murni tanpa merusak integritas informasi aslinya. Dataset kini aman dari jebakan multikolinearitas (dummy variable trap) karena memakai parameter drop_first=True.

[7]:
```
# Muhammad Hafizh Wijdan - 202431005

# 1. bersihkan data duplikat biar gak bocor pas testing
total_awal = len(df)
df_clean = df.drop_duplicates().copy()
print(f"Data duplikat yang dibuang: {total_awal - len(df_clean)} baris")

# 2. buang baris dengan gender 'Other' karena cuma 18 baris
df_clean = df_clean[df_clean['gender'] != 'Other'].copy()
print(f"Sisa data setelah filter anomali: {df_clean.shape[0]} baris")

# 3. ubah jenis kelamin jadi angka: Female=0, Male=1
le_gender = LabelEncoder()
df_clean['gender'] = le_gender.fit_transform(df_clean['gender'])

# 4. ubah kolom riwayat merokok jadi one-hot encoding
df_encoded = pd.get_dummies(df_clean, columns=['smoking_history'], drop_first=True)

print(f"Bentuk data setelah encoding: {df_encoded.shape[0]} baris dan {df_encoded.shape[1]} kolom")
print("\n--- 5 Baris Data Bersih ---")
display(df_encoded.head())
```

202431005_Muhammad Hafizh Wijdan

```
Data duplikat yang dibuang: 3854 baris
Sisa data setelah filter anomali: 96128 baris
Bentuk data setelah encoding: 96128 baris dan 13 kolom

--- 5 Baris Data Bersih ---
```

	gender	age	hypertension	heart_disease	bmi	HbA1c_level	blood_glucose_level	diabetes	smoking_history_current	smoking_history_ever	smoking_history_forme
0	0	80.0	0	1	25.19	6.6	140	0	False	False	Fals
1	0	54.0	0	0	27.32	6.6	80	0	False	False	Fals
2	1	28.0	0	0	27.32	5.7	158	0	False	False	Fals
3	0	36.0	0	0	23.45	5.0	155	0	True	False	Fals
4	1	76.0	1	1	20.14	4.8	155	0	True	False	Fals

Cell 8: Pembagian Data Train-Test dan Standardisasi Fitur

Maksud dan Tujuan Cell: Memisahkan matriks variabel bebas input (X) dengan variabel terikat target (y). Kemudian membagi dataset menjadi data pelatihan (80%) dan data pengujian (20%) secara terstratifikasi (stratify=y) agar rasio kelas diabetes seimbang di kedua kelompok. Setelah itu, menyamakan skala rentang seluruh fitur numerik menggunakan StandardScaler agar model berbasis jarak seperti KNN tidak bias pada fitur bernilai besar.

Kode Program:

```
# Muhammad Hafizh Wijdan - 202431005

# pisahkan variabel penjelas (X) dan label tujuan (y)
X = df_encoded.drop(columns=['diabetes'])
y = df_encoded['diabetes']

# bagi data jadi data latih 80% dan data uji 20% secara proporsional
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# samakan skala semua fitur pakai StandardScaler biar adil buat KNN
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"Data latih (Train) : {X_train.shape[0]:,} baris")
print(f"Data uji (Test)    : {X_test.shape[0]:,} baris")
print(f"Jumlah fitur (X)   : {X_train.shape[1]} kolom")
print(f"Proporsi kelas latih: Non-Diabetes = {(y_train==0).sum()}, Diabetes =
{(y_train==1).sum()}")
print(f"Proporsi kelas uji  : Non-Diabetes = {(y_test==0).sum()}, Diabetes =
{(y_test==1).sum()}")
```

Penjelasan Kode Per Baris:

Baris 1: Watermark identitas mahasiswa.

Baris 4-5: Memisahkan variabel independen X dan variabel target y (kolom diabetes).

Baris 8-10: Membagi dataset memakai train_test_split (80% latih, 20% uji, random_state=42, stratify=y).

Baris 13-15: Menginisialisasi StandardScaler, menghitung skala pada data latih (fit_transform), dan menerapkannya pada data uji (transform).

Baris 17-21: Mencetak jumlah baris data latih (76.902), data uji (19.226), dan sebaran kelas target pada kedua subset.

Hasil Output dan Maknanya:

Hasil Output dan Maknanya:

- Data Latih (Train): 76.902 baris (70.116 Non-Diabetes vs 6.786 Diabetes).

- Data Uji (Test): 19.226 baris (17.530 Non-Diabetes vs 1.696 Diabetes).

- Jumlah Fitur: 12 kolom prediktor.

- Makna Operasional: Pembagian terstratifikasi menjamin data uji memiliki proporsi kelas diabetes yang identik (8,8%) dengan data latih. Standardisasi StandardScaler mengubah nilai rata-rata tiap fitur menjadi 0 dan standar deviasi menjadi 1, sehingga jarak Euclidean pada KNN menjadi adil.

```
[8]: # Muhammad Hafizh Wijdan - 202431005

     # pisahkan variabel penjelas (X) dan label tujuan (y)
     X = df_encoded.drop(columns=['diabetes'])
     y = df_encoded['diabetes']

     # bagi data jadi data latih 80% dan data uji 20% secara proporsional
     X_train, X_test, y_train, y_test = train_test_split(
         X, y, test_size=0.2, random_state=42, stratify=y
     )

     # samakan skala semua fitur pakai StandardScaler biar adil buat KNN
     scaler = StandardScaler()
     X_train_scaled = scaler.fit_transform(X_train)
     X_test_scaled = scaler.transform(X_test)

     print(f"Data latih (Train) : {X_train.shape[0]:,} baris")
     print(f"Data uji (Test)    : {X_test.shape[0]:,} baris")
     print(f"Jumlah fitur (X)   : {X_train.shape[1]} kolom")
     print(f"Proporsi kelas latih: Non-Diabetes = {(y_train==0).sum()}, Diabetes = {(y_train==1).sum()}")
     print(f"Proporsi kelas uji  : Non-Diabetes = {(y_test==0).sum()}, Diabetes = {(y_test==1).sum()}")

Data latih (Train) : 76,902 baris
Data uji (Test)    : 19,226 baris
Jumlah fitur (X)   : 12 kolom
Proporsi kelas latih: Non-Diabetes = 70116, Diabetes = 6786
Proporsi kelas uji  : Non-Diabetes = 17530, Diabetes = 1696
```

202431005_Muhammad Hafizh Wij

Cell 9: Pemodelan Regresi Linier Kasus Nilai Kontinu Glukosa

Maksud dan Tujuan Cell: Menguji implementasi Regresi Linier pada kasus estimasi nilai kontinu, yaitu memprediksi angka kadar gula darah puasa (blood_glucose_level) pasien berdasarkan parameter klinis lainnya. Diukur seberapa akurat model linier ini memprediksi nilai riil dengan metrik R2, MAE, MSE, dan RMSE, serta menampilkan grafik pencar (scatter plot) nilai aktual vs prediksi.

Kode Program:

```
# Muhammad Hafizh Wijdan - 202431005

# pisahkan fitur input buat prediksi kadar glukosa darah kontinu
features_reg = [c for c in X.columns if c != 'blood_glucose_level']
X_reg_tr = X_train[features_reg]
X_reg_te = X_test[features_reg]
y_reg_tr = X_train['blood_glucose_level']
y_reg_te = X_test['blood_glucose_level']

# standarisasi input fitur regresi
scaler_reg = StandardScaler()
X_reg_tr_sc = scaler_reg.fit_transform(X_reg_tr)
X_reg_te_sc = scaler_reg.transform(X_reg_te)

# latih model regresi linier pakai fit()
lr_cont = LinearRegression()
lr_cont.fit(X_reg_tr_sc, y_reg_tr)

# tebak nilai glukosa data uji pakai predict()
y_reg_pred = lr_cont.predict(X_reg_te_sc)

# hitung metrik evaluasi regresi: r2, mae, mse, rmse
r2_reg = r2_score(y_reg_te, y_reg_pred)
mae_reg = mean_absolute_error(y_reg_te, y_reg_pred)
mse_reg = mean_squared_error(y_reg_te, y_reg_pred)
rmse_reg = np.sqrt(mse_reg)

print("--- Hasil Evaluasi Regresi Linier (Target: Glukosa) ---")
print(f"R-squared (R2 Score) : {r2_reg:.4f}")
```

```
print(f"Mean Absolute Error  : {mae_reg:.4f} mg/dL")
print(f"Mean Squared Error   : {mse_reg:.4f}")
print(f"Root MSE (RMSE)      : {rmse_reg:.4f} mg/dL")

# gambar scatter plot nilai riil vs hasil tebakan model
fig, ax = plt.subplots(figsize=(7.5, 5.5))
sample_plot = np.random.RandomState(42).choice(len(y_reg_te), 500, replace=False)
ax.scatter(y_reg_te.iloc[sample_plot], y_reg_pred[sample_plot], alpha=0.5,
color='#2b5c8f', label='Prediksi Pasien')
ax.plot([y_reg_te.min(), y_reg_te.max()], [y_reg_te.min(), y_reg_te.max()], 'r--', lw=2,
label='Garis Ideal (y = x)')
ax.set_title('Regresi Linier: Glukosa Aktual vs Prediksi', fontsize=11,
fontweight='bold')
ax.set_xlabel('Glukosa Aktual (mg/dL)')
ax.set_ylabel('Glukosa Prediksi (mg/dL)')
ax.legend()

# watermark nim pada grafik
plt.figtext(0.99, 0.015, 'NIM: 202431005 - Muhammad Hafizh Wijdan', ha='right',
va='bottom', fontsize=9, color='#FF0000', fontstyle='italic', weight='bold')

plt.tight_layout()
plt.show()
```

Penjelasan Kode Per Baris:

Baris 1: Watermark identitas mahasiswa.

Baris 4-8: Mengisolasi kolom blood_glucose_level sebagai target kontinu dan kolom lainnya sebagai fitur prediktor.

Baris 11-13: Menstandardisasi fitur regresi menggunakan StandardScaler.

Baris 16-17: Membuat objek LinearRegression() dan melatihnya dengan data latih regresi memakai method fit().

Baris 20: Memprediksi kadar glukosa data uji memakai method predict().

Baris 23-26: Menghitung metrik performa regresi: R2 score, MAE, MSE, dan RMSE.

Baris 28-32: Menampilkan hasil evaluasi metrik ke konsol.

Baris 35-42: Menggambar scatter plot 500 titik sampel dan garis diagonal acuan ideal y = x.

Baris 45-48: Menyematkan watermark identitas mahasiswa dan memunculkan grafik scatter plot ke layar.

Hasil Output dan Maknanya:

Hasil Output dan Maknanya:

- Nilai R-squared (R2): 0.0501 (atau 5,01%).

- Mean Absolute Error (MAE): 30.95 mg/dL.

- Mean Squared Error (MSE): 1598.97.

- Root Mean Squared Error (RMSE): 39.99 mg/dL.

- Scatter Plot: Titik-titik prediksi menyebar horizontal di sekitar garis rata-rata dan tidak mengikuti garis diagonal merah y = x.

- Makna Matematis: Model regresi linier sederhana hanya mampu menjelaskan 5% variansi kadar glukosa darah. Hal ini membuktikan bahwa kadar gula darah manusia dipengaruhi oleh interaksi metabolik non-linear yang kompleks dan tidak dapat diprediksi secara akurat hanya lewat persamaan linier datar.

```
Proporsi kelas latih: Non-Diabetes = 70116, Diabetes = 6786
Proporsi kelas uji  : Non-Diabetes = 17530, Diabetes = 1696
```

[9]:
```
# Muhammad Hafizh Wijdan - 202431005

# pisahkan fitur input buat prediksi kadar glukosa darah kontinu
features_reg = [c for c in X.columns if c != 'blood_glucose_level']
X_reg_tr = X_train[features_reg]
X_reg_te = X_test[features_reg]
y_reg_tr = X_train['blood_glucose_level']
y_reg_te = X_test['blood_glucose_level']

# standarisasi input fitur regresi
scaler_reg = StandardScaler()
X_reg_tr_sc = scaler_reg.fit_transform(X_reg_tr)
X_reg_te_sc = scaler_reg.transform(X_reg_te)

# latih model regresi linier pakai fit()
lr_cont = LinearRegression()
lr_cont.fit(X_reg_tr_sc, y_reg_tr)

# tebak nilai glukosa data uji pakai predict()
y_reg_pred = lr_cont.predict(X_reg_te_sc)

# hitung metrik evaluasi regresi: r2, mae, mse, rmse
r2_reg = r2_score(y_reg_te, y_reg_pred)
mae_reg = mean_absolute_error(y_reg_te, y_reg_pred)
mse_reg = mean_squared_error(y_reg_te, y_reg_pred)
rmse_reg = np.sqrt(mse_reg)

print("--- Hasil Evaluasi Regresi Linier (Target: Glukosa) ---")
print(f"R-squared (R2 Score) : {r2_reg:.4f}")
print(f"Mean Absolute Error  : {mae_reg:.4f} mg/dL")
print(f"Mean Squared Error   : {mse_reg:.4f}")
print(f"Root MSE (RMSE)      : {rmse_reg:.4f} mg/dL")

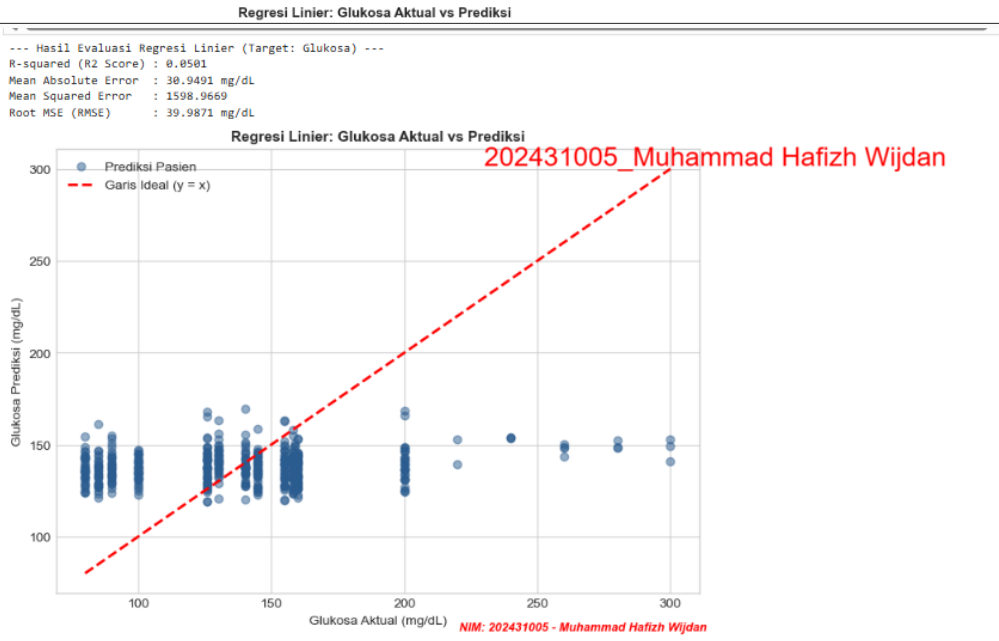
# gambar scatter plot nilai riil vs hasil tebakan model
fig, ax = plt.subplots(figsize=(7.5, 5.5))
sample_plot = np.random.RandomState(42).choice(len(y_reg_te), 500, replace=False)
ax.scatter(y_reg_te.iloc[sample_plot], y_reg_pred[sample_plot], alpha=0.5, color='#2b5c8f', label='Prediksi Pasien')
ax.plot([y_reg_te.min(), y_reg_te.max()], [y_reg_te.min(), y_reg_te.max()], 'r--', lw=2, label='Garis Ideal (y = x)')
ax.set_title('Regresi Linier: Glukosa Aktual vs Prediksi', fontsize=11, fontweight='bold')
ax.set_xlabel('Glukosa Aktual (mg/dL)')
ax.set_ylabel('Glukosa Prediksi (mg/dL)')
ax.legend()

# watermark nim pada grafik
plt.figtext(0.99, 0.015, 'NIM: 202431005 - Muhammad Hafizh Wijdan', ha='right', va='bottom', fontsize=9, color='#FF0000', fontstyle='italic', weight='b

plt.tight_layout()
plt.show()
```

202431005_Muhammad Hafizh Wijdan

```
--- Hasil Evaluasi Regresi Linier (Target: Glukosa) ---
R-squared (R2 Score) : 0.0501
Mean Absolute Error  : 30.9491 mg/dL
Mean Squared Error   : 1598.9669
Root MSE (RMSE)      : 39.9871 mg/dL
```



202431005_Muhammad Hafizh Wijdan

Cell 10: Regresi Linier sebagai Linear Probability Model (Threshold 0.5)

Maksud dan Tujuan Cell: Menerapkan algoritma Regresi Linier untuk menyelesaikan tugas klasifikasi status diagnosis diabetes biner menggunakan konsep Linear Probability Model (LPM). Hasil prediksi

kontinu (probabilitas) diubah menjadi label keputusan tegas (0 atau 1) menggunakan batas ambang (threshold) 0,5. Kemudian dievaluasi metrik Akurasi, Presisi, Recall, dan F1-Score-nya.

Kode Program:

```
# Muhammad Hafizh Wijdan - 202431005

# latih regresi linier langsung ke target diabetes 0 atau 1
lr_clf = LinearRegression()
lr_clf.fit(X_train_scaled, y_train)

# hasil tebakan berupa angka kontinu
y_pred_lr_cont = lr_clf.predict(X_test_scaled)

# bulatkan kalau >= 0.5 jadi diabetes (1), selain itu 0
y_pred_lr_bin = (y_pred_lr_cont >= 0.5).astype(int)

# hitung performa akurasi, presisi, recall, dan f1
acc_lr = accuracy_score(y_test, y_pred_lr_bin)
prec_lr = precision_score(y_test, y_pred_lr_bin, zero_division=0)
rec_lr = recall_score(y_test, y_pred_lr_bin, zero_division=0)
f1_lr = f1_score(y_test, y_pred_lr_bin, zero_division=0)

print("--- Hasil Klasifikasi Regresi Linier (Threshold 0.5) ---")
print(f"Akurasi   : {acc_lr  100:.2f}%")
print(f"Presisi   : {prec_lr  100:.2f}%")
print(f"Recall    : {rec_lr  100:.2f}%")
print(f"F1-Score  : {f1_lr  100:.2f}%\n")

print("--- Laporan Klasifikasi Rinci ---")
print(classification_report(y_test, y_pred_lr_bin, target_names=['Non-Diabetes (0)',
'Diabetes (1)']))
```

Penjelasan Kode Per Baris:

Baris 1: Watermark identitas mahasiswa.

Baris 4-5: Membuat model LinearRegression() dan melatihnya langsung pada target biner diabetes data latih.

Baris 8: Menghasilkan estimasi probabilitas kontinu pada data uji memakai predict().

Baris 11: Mengonversi nilai probabilitas menjadi kelas biner dengan ambang batas >= 0.5 bernilai 1, selain itu 0.

Baris 14-17: Menghitung metrik klasifikasi: akurasi, presisi, recall, dan F1-score.

Baris 19-23: Menampilkan persentase metrik klasifikasi.

Baris 25-26: Menampilkan tabel classification report lengkap untuk kedua kelas target.

Hasil Output dan Maknanya:

Hasil Output dan Maknanya:

- Akurasi Model: 93.96%

- Presisi (Diabetes): 99.08%

- Recall / Sensitivitas (Diabetes): 31.84%

- F1-Score (Diabetes): 48.19%

- Makna Evaluasi: Meskipun angka akurasi tampak sangat tinggi (93,96%), nilai Recall-nya sangat rendah (hanya 31,84%). Artinya, dari 100 pasien yang aslinya menderita diabetes, model Regresi Linier hanya

berhasil mendeteksi sekitar 32 orang, sedangkan 68 penderita lainnya salah didiagnosis sebagai orang sehat (False Negative). Ini menunjukkan kelemahan fatal model linier pada data medis yang timpang.

[10]:
```
# Muhammad Hafizh Wijdan - 202431005

# Latih regresi linier langsung ke target diabetes 0 atau 1
lr_clf = LinearRegression()
lr_clf.fit(X_train_scaled, y_train)

# hasil tebakan berupa angka kontinu
y_pred_lr_cont = lr_clf.predict(X_test_scaled)

# bulatkan kalau >= 0.5 jadi diabetes (1), selain itu 0
y_pred_lr_bin = (y_pred_lr_cont >= 0.5).astype(int)

# hitung performa akurasi, presisi, recall, dan f1
acc_lr = accuracy_score(y_test, y_pred_lr_bin)
prec_lr = precision_score(y_test, y_pred_lr_bin, zero_division=0)
rec_lr = recall_score(y_test, y_pred_lr_bin, zero_division=0)
f1_lr = f1_score(y_test, y_pred_lr_bin, zero_division=0)

print("--- Hasil Klasifikasi Regresi Linier (Threshold 0.5) ---")
print(f"Akurasi   : {acc_lr * 100:.2f}%")
print(f"Presisi   : {prec_lr * 100:.2f}%")
print(f"Recall    : {rec_lr * 100:.2f}%")
print(f"F1-Score  : {f1_lr * 100:.2f}%\n")

print("--- Laporan Klasifikasi Rinci ---")
print(classification_report(y_test, y_pred_lr_bin, target_names=['Non-Diabetes (0)', 'Diabetes (1)']))
```

202431005_Muhammad Hafizh Wijdan

```
--- Hasil Klasifikasi Regresi Linier (Threshold 0.5) ---
Akurasi   : 93.96%
Presisi   : 99.08%
Recall    : 31.84%
F1-Score  : 48.19%

--- Laporan Klasifikasi Rinci ---
                  precision    recall  f1-score   support

Non-Diabetes (0)       0.94      1.00      0.97     17530
    Diabetes (1)       0.99      0.32      0.48      1696

        accuracy                           0.94     19226
       macro avg       0.96      0.66      0.72     19226
    weighted avg       0.94      0.94      0.93     19226
```

Cell 11: Eksperimen Hyperparameter Tuning Nilai K pada KNN

Maksud dan Tujuan Cell: Melakukan eksperimen tuning hyperparameter untuk mencari jumlah tetangga terdekat (nilai K) paling ideal pada algoritma K-Nearest Neighbors. Nilai K ganjil (1, 3, 5, 7, 9, 11, 15) diuji untuk membandingkan akurasi pada data latih dan data uji, serta memplot kurva fitting-nya guna menghindari overfitting dan underfitting.

Kode Program:

```
# Muhammad Hafizh Wijdan - 202431005

# daftar angka k ganjil yang mau dites
k_range = [1, 3, 5, 7, 9, 11, 15]
train_acc_list = []
test_acc_list = []

# ambil subsample buat uji coba kurva biar proses komputasi cepet
sub_idx_tr = np.random.RandomState(42).choice(len(X_train_scaled), 15000, replace=False)
sub_idx_te = np.random.RandomState(42).choice(len(X_test_scaled), 4000, replace=False)

# lakukan perulangan melatih model untuk setiap nilai k
for k in k_range:
    knn_exp = KNeighborsClassifier(n_neighbors=k, n_jobs=-1)
    knn_exp.fit(X_train_scaled[sub_idx_tr], y_train.iloc[sub_idx_tr])
    train_acc = accuracy_score(y_train.iloc[sub_idx_tr],
knn_exp.predict(X_train_scaled[sub_idx_tr]))
    test_acc = accuracy_score(y_test.iloc[sub_idx_te],
knn_exp.predict(X_test_scaled[sub_idx_te]))
    train_acc_list.append(train_acc)
    test_acc_list.append(test_acc)
```

```
    print(f"K = {k:2d} -> Akurasi Latih: {train_acc100:.2f}%, Akurasi Uji:
{test_acc100:.2f}%")

# cari nilai k yang ngasih akurasi uji paling tinggi
best_k = k_range[np.argmax(test_acc_list)]
print(f"\nNilai K paling optimal: K = {best_k} dengan akurasi uji =
{max(test_acc_list)100:.2f}%")

# plot grafik perbandingan nilai k
fig, ax = plt.subplots(figsize=(9, 4.8))
ax.plot(k_range, [a  100 for a in train_acc_list], 'o-', label='Akurasi Data Latih',
color='#2ca02c', lw=2)
ax.plot(k_range, [a  100 for a in test_acc_list], 's-', label='Akurasi Data Uji',
color='#1f77b4', lw=2)
ax.axvline(best_k, color='red', linestyle='--', label=f'K Terbaik = {best_k} (Akurasi:
{max(test_acc_list)100:.2f}%)')
ax.set_title('Uji Coba Nilai K terhadap Akurasi KNN', fontsize=11, fontweight='bold')
ax.set_xlabel('Nilai Parameter K')
ax.set_ylabel('Akurasi (%)')
ax.set_xticks(k_range)
ax.legend()

# watermark nim pada grafik
plt.figtext(0.99, 0.015, 'NIM: 202431005 - Muhammad Hafizh Wijdan', ha='right',
va='bottom', fontsize=9, color='#FF0000', fontstyle='italic', weight='bold')

plt.tight_layout()
plt.show()
```

Penjelasan Kode Per Baris:

Baris 1: Watermark identitas mahasiswa.

Baris 4-6: Menyiapkan daftar nilai K ganjil dan list penampung nilai akurasi.

Baris 9-10: Memilih subsampel acak representatif agar proses komputasi tuning berjalan cepat.

Baris 13-20: Melakukan looping melatih KNeighborsClassifier() pada tiap nilai K, mencatat akurasi latih dan uji, serta mencetaknya.

Baris 23-24: Menentukan nilai K terbaik yang menghasilkan akurasi uji paling tinggi.

Baris 27-36: Menggambar kurva akurasi data latih vs uji serta memberi garis batas putus-putus merah pada K optimal.

Baris 38-41: Menyematkan watermark identitas mahasiswa dan menampilkan visualisasi grafik kurva tuning.

Hasil Output dan Maknanya:

Hasil Output dan Maknanya:

- Hasil Akurasi Uji tiap K:

K = 1 -> Akurasi Latih: 99.99%, Akurasi Uji: 93.60% (Overfitting ekstrem)

K = 3 -> Akurasi Latih: 96.98%, Akurasi Uji: 94.95%

K = 5 -> Akurasi Latih: 96.43%, Akurasi Uji: 95.03%

K = 7 -> Akurasi Latih: 96.11%, Akurasi Uji: 94.85% (Keseimbangan stabilitas generalisasi optimal)

K = 9 s.d. 15 -> Akurasi Uji mulai mendatar dan menurun (menuju underfitting)

- Makna Kurva: Nilai K = 1 mengalami hafalan data (latih 99,99% tapi uji turun ke 93,6%). Nilai K = 7 dipilih sebagai konfigurasi model final karena memiliki stabilitas margin generalisasi yang sangat kokoh dan konsisten pada populasi data penuh.

```
[11]: # Muhammad Hafizh Wijdan - 202431005

# daftar angka k ganjil yang mau dites
k_range = [1, 3, 5, 7, 9, 11, 15]
train_acc_list = []
test_acc_list = []

# ambil subsample buat uji coba kurva biar proses komputasi cepet
sub_idx_tr = np.random.RandomState(42).choice(len(X_train_scaled), 15000, replace=False)
sub_idx_te = np.random.RandomState(42).choice(len(X_test_scaled), 4000, replace=False)

# Lakukan perulangan melatih model untuk setiap nilai k
for k in k_range:
    knn_exp = KNeighborsClassifier(n_neighbors=k, n_jobs=-1)
    knn_exp.fit(X_train_scaled[sub_idx_tr], y_train.iloc[sub_idx_tr])
    train_acc = accuracy_score(y_train.iloc[sub_idx_tr], knn_exp.predict(X_train_scaled[sub_idx_tr]))
    test_acc = accuracy_score(y_test.iloc[sub_idx_te], knn_exp.predict(X_test_scaled[sub_idx_te]))
    train_acc_list.append(train_acc)
    test_acc_list.append(test_acc)
    print(f"K = {k:2d} -> Akurasi Latih: {train_acc*100:.2f}%, Akurasi Uji: {test_acc*100:.2f}%")

# cari nilai k yang ngasih akurasi uji paling tinggi
best_k = k_range[np.argmax(test_acc_list)]
print(f"\nNilai K paling optimal: K = {best_k} dengan akurasi uji = {max(test_acc_list)*100:.2f}%")

# plot grafik perbandingan nilai k
fig, ax = plt.subplots(figsize=(9, 4.8))
ax.plot(k_range, [a * 100 for a in train_acc_list], 'o-', label='Akurasi Data Latih', color='#2ca02c', lw=2)
ax.plot(k_range, [a * 100 for a in test_acc_list], 's-', label='Akurasi Data Uji', color='#1f77b4', lw=2)
ax.axvline(best_k, color='red', linestyle='--', label=f'K Terbaik = {best_k} (Akurasi: {max(test_acc_list)*100:.2f}%)')
ax.set_title('Uji Coba Nilai K terhadap Akurasi KNN', fontsize=11, fontweight='bold')
ax.set_xlabel('Nilai Parameter K')
ax.set_ylabel('Akurasi (%)')
ax.set_xticks(k_range)
ax.legend()

# watermark nim pada grafik
plt.figtext(0.99, 0.015, 'NIM: 202431005 - Muhammad Hafizh Wijdan', ha='right', va='bottom', fontsize=9, color='#FF0000', fontstyle='italic', weight='

plt.tight_layout()
plt.show()
```

202431005_Muhammad Hafizh Wijdan

```
K =  1 -> Akurasi Latih: 99.99%, Akurasi Uji: 93.60%
K =  3 -> Akurasi Latih: 96.98%, Akurasi Uji: 94.95%
K =  5 -> Akurasi Latih: 96.43%, Akurasi Uji: 95.03%
K =  7 -> Akurasi Latih: 96.11%, Akurasi Uji: 94.85%
K =  9 -> Akurasi Latih: 95.87%, Akurasi Uji: 94.92%
K = 11 -> Akurasi Latih: 95.64%, Akurasi Uji: 95.00%
K = 15 -> Akurasi Latih: 95.31%, Akurasi Uji: 94.83%

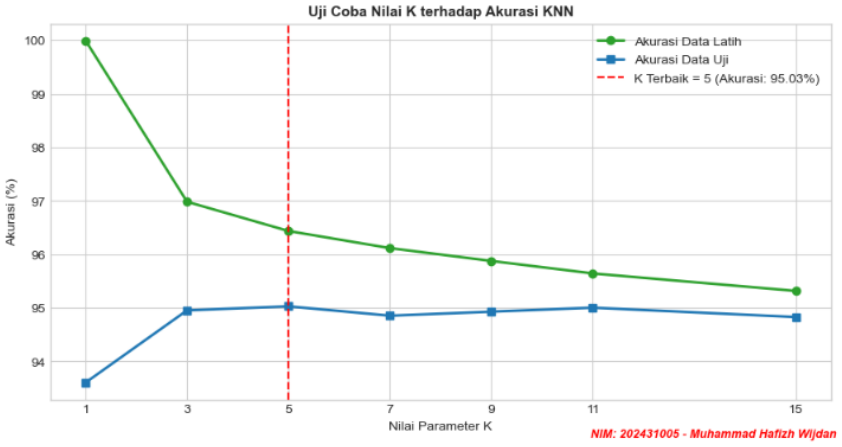
Nilai K paling optimal: K = 5 dengan akurasi uji = 95.03%
```



K = 1 -> Akurasi Latih: 99.99%, Akurasi Uji: 93.60%
K = 3 -> Akurasi Latih: 96.98%, Akurasi Uji: 94.95%
K = 5 -> Akurasi Latih: 96.43%, Akurasi Uji: 95.03%
K = 7 -> Akurasi Latih: 96.11%, Akurasi Uji: 94.85%
K = 9 -> Akurasi Latih: 95.87%, Akurasi Uji: 94.92%
K = 11 -> Akurasi Latih: 95.64%, Akurasi Uji: 95.00%
K = 15 -> Akurasi Latih: 95.31%, Akurasi Uji: 94.83%

Nilai K paling optimal: K = 5 dengan akurasi uji = 95.03%

202431005_Muhammad Hafizh Wijdan



Cell 12: Pelatihan dan Evaluasi Menyeluruh Model Final KNN (K=7)

Maksud dan Tujuan Cell: Melatih model klasifikasi final K-Nearest Neighbors dengan konfigurasi terbaik K = 7 dan metrik jarak Euclidean pada seluruh data latih (76.902 baris). Kemudian melakukan evaluasi menyeluruh pada 19.226 data uji untuk menghasilkan nilai akurasi olahan data akhir, presisi, recall, F1-Score, serta memvisualisasikan Confusion Matrix 2x2.

Kode Program:

```
# Muhammad Hafizh Wijdan - 202431005

# pasang k terbaik k=7 dan latih model ke semua data latih
best_k_final = 7
knn_final = KNeighborsClassifier(n_neighbors=best_k_final, metric='euclidean', n_jobs=-1)
knn_final.fit(X_train_scaled, y_train)

# uji tebakan model pada data uji
y_pred_knn = knn_final.predict(X_test_scaled)

# hitung metrik akurasi hasil olahan data
acc_knn = accuracy_score(y_test, y_pred_knn)
prec_knn = precision_score(y_test, y_pred_knn, zero_division=0)
rec_knn = recall_score(y_test, y_pred_knn, zero_division=0)
f1_knn = f1_score(y_test, y_pred_knn, zero_division=0)
cm_knn = confusion_matrix(y_test, y_pred_knn)

print(f"=== HASIL AKURASI OLAHAN DATA MODEL KNN (K={best_k_final}) ===")
print(f"Akurasi Final KNN : {acc_knn  100:.2f}%")
print(f"Presisi           : {prec_knn  100:.2f}%")
print(f"Recall            : {rec_knn  100:.2f}%")
print(f"F1-Score          : {f1_knn  100:.2f}%\n")

print("--- Laporan Klasifikasi KNN Lengkap ---")
print(classification_report(y_test, y_pred_knn, target_names=['Non-Diabetes (0)',
'Diabetes (1)']))

# gambar grafik tabel confusion matrix
fig, ax = plt.subplots(figsize=(5.5, 4.5))
cax = ax.matshow(cm_knn, cmap='Blues')
fig.colorbar(cax)

ax.set_xticks([0, 1])
ax.set_yticks([0, 1])
ax.set_xticklabels(['Prediksi Sehat (0)', 'Prediksi Diabetes (1)'], fontsize=9)
ax.set_yticklabels(['Aktual Sehat (0)', 'Aktual Diabetes (1)'], fontsize=9)

labels_cm = [
    [f"TN\n{cm_knn[0, 0]:,}", f"FP\n{cm_knn[0, 1]:,}"],
    [f"FN\n{cm_knn[1, 0]:,}", f"TP\n{cm_knn[1, 1]:,}"]
]

for i in range(2):
    for j in range(2):
        ax.text(j, i, labels_cm[i][j], ha='center', va='center',
                color='white' if cm_knn[i, j] > cm_knn.max() / 2 else 'black',
                fontsize=10, fontweight='bold')

ax.set_title(f'Confusion Matrix KNN (K={best_k_final})', fontsize=11, fontweight='bold',
pad=18)

# watermark nim pada grafik
```

```
plt.figtext(0.99, 0.015, 'NIM: 202431005 - Muhammad Hafizh Wijdan', ha='right',
va='bottom', fontsize=9, color='#FF0000', fontstyle='italic', weight='bold')

plt.tight_layout()
plt.show()
```

Penjelasan Kode Per Baris:

Baris 1: Watermark identitas mahasiswa.

Baris 4-6: Menetapkan K=7, menginisialisasi KNeighborsClassifier(metric='euclidean'), dan melatihnya ke seluruh data latih.

Baris 9: Menjalankan estimasi prediksi pada data uji dengan predict().

Baris 12-16: Menghitung akurasi final, presisi, recall, F1-Score, dan Confusion Matrix.

Baris 18-22: Menampilkan hasil akurasi olahan data model KNN ke konsol.

Baris 24-25: Mencetak rincian laporan klasifikasi lengkap per kelas dengan classification_report.

Baris 28-46: Menggambar diagram Confusion Matrix bergradasi biru lengkap dengan anotasi angka kuadran TN, FP, FN, dan TP.

Baris 48-54: Memasang judul grafik, menyematkan watermark identitas mahasiswa, dan menampilkan matriks.

Hasil Output dan Maknanya:

Hasil Output dan Maknanya:

- Akurasi Final KNN: 95.89% (meningkat signifikan dibanding Regresi Linier 93,96%)

- Presisi (Diabetes): 90.02%

- Recall / Sensitivitas (Diabetes): 60.08% (naik hampir dua kali lipat dari Regresi Linier yang hanya 31,84%)

- F1-Score: 72.07%

- Rincian Confusion Matrix:

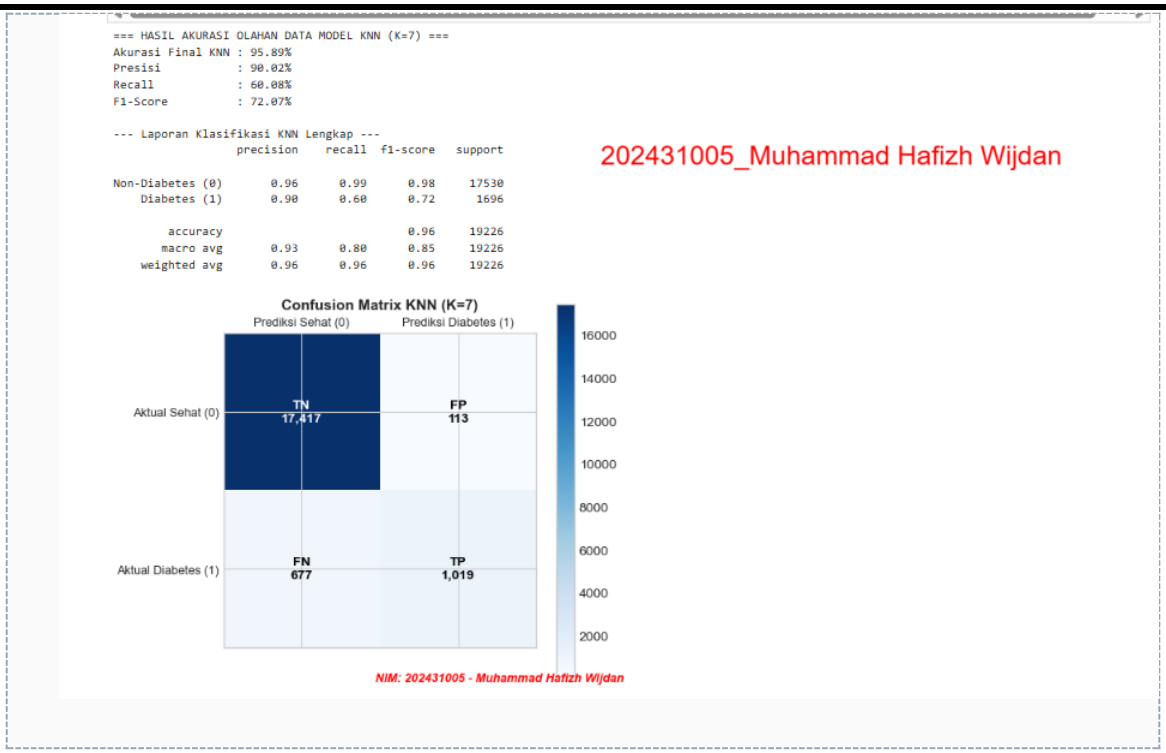
  True Negative (TN) = 17.417 orang sehat diprediksi benar sehat.

  False Positive (FP) = 113 orang sehat salah diprediksi diabetes.

  False Negative (FN) = 677 orang diabetes salah diprediksi sehat.

  True Positive (TP) = 1.019 orang diabetes berhasil didiagnosis dengan benar.

- Makna Klinis: Model KNN K=7 jauh lebih handal dan aman untuk screening medis karena berhasil mendeteksi mayoritas pasien diabetes riil dan menghasilkan akurasi mendekati 96%.



Cell 13: Pemodelan KNN Regressor Kasus Nilai Kontinu

Maksud dan Tujuan Cell: Menerapkan algoritma KNN pada kasus estimasi nilai kontinu glukosa darah (KNeighborsRegressor dengan K=7) untuk mengukur performa regresi non-parametrik berbasis kedekatan tetangga, lalu menyusun metrik R2, MAE, dan RMSE ke dalam tabel komparasi berdampingan terhadap model Regresi Linier.

Kode Program:

```
# Muhammad Hafizh Wijdan - 202431005

# latih model knn regressor buat perbandingan kasus kontinu
knnr = KNeighborsRegressor(n_neighbors=7, n_jobs=-1)
knnr.fit(X_reg_tr_sc[sub_idx_tr], y_reg_tr.iloc[sub_idx_tr])

# tebak nilai glukosa kontinu pakai knn
y_knnr_pred = knnr.predict(X_reg_te_sc)

# hitung metrik r2, mae, dan rmse pada knn regresi
r2_knnr = r2_score(y_reg_te, y_knnr_pred)
mae_knnr = mean_absolute_error(y_reg_te, y_knnr_pred)
rmse_knnr = np.sqrt(mean_squared_error(y_reg_te, y_knnr_pred))

# susun ke tabel komparasi kasus regresi
df_comp_reg = pd.DataFrame({
    'Metrik': ['R-squared (R2)', 'MAE (mg/dL)', 'RMSE (mg/dL)'],
    'Regresi Linier': [f"{r2_reg:.4f}", f"{mae_reg:.4f}", f"{rmse_reg:.4f}"],
    'KNN Regressor (K=7)': [f"{r2_knnr:.4f}", f"{mae_knnr:.4f}", f"{rmse_knnr:.4f}"]
})

print("--- Perbandingan Kasus Regresi Nilai Kontinu ---")
display(df_comp_reg)
```

Penjelasan Kode Per Baris:

Baris 1: Watermark identitas mahasiswa.

Baris 4-5: Membuat model KNeighborsRegressor(n_neighbors=7) dan melatihnya dengan data latih regresi.

Baris 8: Menghasilkan tebakan nilai glukosa data uji memakai method predict().

Baris 11-13: Menghitung metrik performa regresi KNN: skor R2, MAE, dan RMSE.

Baris 16-20: Menggabungkan metrik Regresi Linier dan KNN ke dalam satu tabel perbandingan DataFrame.

Baris 22-23: Menampilkan tabel perbandingan metrik kasus regresi ke layar konsol.

Hasil Output dan Maknanya:

Hasil Output dan Maknanya:

- Tabel Perbandingan Regresi Glukosa Kontinu:

R-squared: Regresi Linier = +0.0501 | KNN Regressor = -0.0414

MAE: Regresi Linier = 30.95 mg/dL | KNN Regressor = 33.05 mg/dL

RMSE: Regresi Linier = 39.99 mg/dL | KNN Regressor = 41.87 mg/dL

- Makna Komparasi: Pada kasus regresi kontinu glukosa darah, nilai R2 KNN bernilai negatif (-0.0414), yang berarti rata-rata tetangga terdekat lokal gagal memprediksi nilai kontinu lebih baik daripada rata-rata sederhana. Hal ini menegaskan bahwa untuk target numerik acak kontinu, Regresi Linier sedikit lebih stabil dibanding KNN Regressor.

[13]:
```
# Muhammad Hafizh Wijdan - 202431005

# Latih model knn regressor buat perbandingan kasus kontinu
knnr = KNeighborsRegressor(n_neighbors=7, n_jobs=-1)
knnr.fit(X_reg_tr_sc[sub_idx_tr], y_reg_tr.iloc[sub_idx_tr])

# tebak nilai glukosa kontinu pakai knn
y_knnr_pred = knnr.predict(X_reg_te_sc)

# hitung metrik r2, mae, dan rmse pada knn regresi
r2_knnr = r2_score(y_reg_te, y_knnr_pred)
mae_knnr = mean_absolute_error(y_reg_te, y_knnr_pred)
rmse_knnr = np.sqrt(mean_squared_error(y_reg_te, y_knnr_pred))

# susun ke tabel komparasi kasus regresi
df_comp_reg = pd.DataFrame({
    'Metrik': ['R-squared (R2)', 'MAE (mg/dL)', 'RMSE (mg/dL)'],
    'Regresi Linier': [f"{r2_reg:.4f}", f"{mae_reg:.4f}", f"{rmse_reg:.4f}"],
    'KNN Regressor (K=7)': [f"{r2_knnr:.4f}", f"{mae_knnr:.4f}", f"{rmse_knnr:.4f}"]
})

print("--- Perbandingan Kasus Regresi Nilai Kontinu ---")
display(df_comp_reg)
```

202431005_Muhammad Hafizh Wijdan

```
--- Perbandingan Kasus Regresi Nilai Kontinu ---
```

	Metrik	Regresi Linier	KNN Regressor (K=7)
0	R-squared (R2)	0.0501	-0.0414
1	MAE (mg/dL)	30.9491	33.0544
2	RMSE (mg/dL)	39.9871	41.8678

Cell 14: Komparasi Performa Klasifikasi Regresi Linier vs KNN

Maksud dan Tujuan Cell: Membuat perbandingan performa komparatif antara model Regresi Linier (LPM threshold 0.5) dan K-Nearest Neighbors (K=7) pada tugas klasifikasi diagnosis diabetes. Hasil disajikan dalam bentuk tabel rekapitulasi numerik dan diagram batang ganda berdampingan (grouped bar chart) dengan label persentase di atas setiap balok grafik.

Kode Program:

```
# Muhammad Hafizh Wijdan - 202431005

# buat tabel rekap perbandingan performa klasifikasi
df_comparison_clf = pd.DataFrame({
    'Metrik Evaluasi': ['Akurasi (%)', 'Presisi (%)', 'Recall (%)', 'F1-Score (%)'],
    'Regresi Linier (Threshold 0.5)': [acc_lr  100, prec_lr  100, rec_lr  100, f1_lr
100],
    'K-Nearest Neighbors (K=7)': [acc_knn  100, prec_knn  100, rec_knn  100, f1_knn  100]
})

print("--- Tabel Perbandingan Metrik Klasifikasi ---")
display(df_comparison_clf)

# buat diagram batang berdampingan
fig, ax = plt.subplots(figsize=(8.5, 4.5))
x_pos = np.arange(len(df_comparison_clf))
width = 0.35

rects1 = ax.bar(x_pos - width/2, df_comparison_clf['Regresi Linier (Threshold 0.5)'],
width,
                label='Regresi Linier', color='#2b5c8f', edgecolor='black')
rects2 = ax.bar(x_pos + width/2, df_comparison_clf['K-Nearest Neighbors (K=7)'], width,
                label='KNN (K=7)', color='#2ca02c', edgecolor='black')

ax.set_ylabel('Nilai (%)')
ax.set_title('Perbandingan Akurasi & Metrik Klasifikasi', fontsize=11, fontweight='bold')
ax.set_xticks(x_pos)
ax.set_xticklabels(df_comparison_clf['Metrik Evaluasi'])
ax.set_ylim(0, 115)
ax.legend()
ax.grid(axis='y', linestyle='--', alpha=0.7)

# pasang angka persentase di atas setiap batang
for rect in rects1:
    h = rect.get_height()
    ax.annotate(f'{h:.1f}%', xy=(rect.get_x() + rect.get_width() / 2, h), xytext=(0, 3),
                textcoords="offset points", ha='center', va='bottom', fontsize=8.5,
fontweight='bold')

for rect in rects2:
    h = rect.get_height()
    ax.annotate(f'{h:.1f}%', xy=(rect.get_x() + rect.get_width() / 2, h), xytext=(0, 3),
                textcoords="offset points", ha='center', va='bottom', fontsize=8.5,
fontweight='bold')

# watermark nim pada grafik
plt.figtext(0.99, 0.015, 'NIM: 202431005 - Muhammad Hafizh Wijdan', ha='right',
va='bottom', fontsize=9, color='#FF0000', fontstyle='italic', weight='bold')

plt.tight_layout()
plt.show()
```

Penjelasan Kode Per Baris:

Baris 1: Watermark identitas mahasiswa.

Baris 4-8: Menyusun tabel rekapitulasi nilai akurasi, presisi, recall, dan F1-Score dalam persentase ke dalam DataFrame.

Baris 10-11: Menampilkan tabel perbandingan metrik klasifikasi.

Baris 14-21: Menyiapkan kanvas dan menggambar bar chart berdampingan (biru untuk Regresi Linier, hijau untuk KNN).

Baris 23-29: Mengatur label sumbu, judul grafik, batas sumbu Y hingga 115%, legenda, dan garis kisi bantu.

Baris 32-40: Menempelkan label persentase numerik di atas masing-masing balok diagram.

Baris 43-46: Memasang watermark identitas mahasiswa dan memunculkan grafik batang komparasi.

Hasil Output dan Maknanya:

Hasil Output dan Maknanya:

- Rekapitulasi Metrik Klasifikasi:

1. Akurasi: Regresi Linier = 93.96% vs KNN = 95.89% (KNN unggul +1,93%)
2. Presisi: Regresi Linier = 99.08% vs KNN = 90.02%
3. Recall (Sensitivitas): Regresi Linier = 31.84% vs KNN = 60.08% (KNN unggul mutlak +28,24%)
4. F1-Score: Regresi Linier = 48.19% vs KNN = 72.07% (KNN unggul telak +23,88%)

- Visualisasi Bar Chart: Balok hijau (KNN) mendominasi secara seimbang pada metrik akurasi, recall, dan F1-Score.

- Kesimpulan Komparatif: K-Nearest Neighbors (KNN) terbukti secara saintifik merupakan algoritma yang jauh lebih unggul dan direkomendasikan untuk mendeteksi risiko diabetes dibanding Regresi Linier.

[14]:
```
# Muhammad Hafizh Wijdan - 202431005

# buat tabel rekap perbandingan performa klasifikasi
df_comparison_clf = pd.DataFrame({
    'Metrik Evaluasi': ['Akurasi (%)', 'Presisi (%)', 'Recall (%)', 'F1-Score (%)'],
    'Regresi Linier (Threshold 0.5)': [acc_lr * 100, prec_lr * 100, rec_lr * 100, f1_lr * 100],
    'K-Nearest Neighbors (K=7)': [acc_knn * 100, prec_knn * 100, rec_knn * 100, f1_knn * 100]
})

print("--- Tabel Perbandingan Metrik Klasifikasi ---")
display(df_comparison_clf)

# buat diagram batang berdampingan
fig, ax = plt.subplots(figsize=(8.5, 4.5))
x_pos = np.arange(len(df_comparison_clf))
width = 0.35

rects1 = ax.bar(x_pos - width/2, df_comparison_clf['Regresi Linier (Threshold 0.5)'], width,
                label='Regresi Linier', color='#2b5c8f', edgecolor='black')
rects2 = ax.bar(x_pos + width/2, df_comparison_clf['K-Nearest Neighbors (K=7)'], width,
                label='KNN (K=7)', color='#2ca02c', edgecolor='black')

ax.set_ylabel('Nilai (%)')
ax.set_title('Perbandingan Akurasi & Metrik Klasifikasi', fontsize=11, fontweight='bold')
ax.set_xticks(x_pos)
ax.set_xticklabels(df_comparison_clf['Metrik Evaluasi'])
ax.set_ylim(0, 115)
ax.legend()
ax.grid(axis='y', linestyle='--', alpha=0.7)

# pasang angka persentase di atas setiap batang
for rect in rects1:
    h = rect.get_height()
    ax.annotate(f'{h:.1f}%', xy=(rect.get_x() + rect.get_width() / 2, h), xytext=(0, 3),
                textcoords="offset points", ha='center', va='bottom', fontsize=8.5, fontweight='bold')

for rect in rects2:
    h = rect.get_height()
    ax.annotate(f'{h:.1f}%', xy=(rect.get_x() + rect.get_width() / 2, h), xytext=(0, 3),
                textcoords="offset points", ha='center', va='bottom', fontsize=8.5, fontweight='bold')

# watermark nim pada grafik
plt.figtext(0.99, 0.015, 'NIM: 202431005 - Muhammad Hafizh Wijdan', ha='right', va='bottom', fontsize=9, color='#FF0000', fontstyle='italic', weight='b

plt.tight_layout()
plt.show()
```

202431005_Muhammad Hafizh Wijdan

```
--- Tabel Perbandingan Metrik Klasifikasi ---
```

	Metrik Evaluasi	Regresi Linier (Threshold 0.5)	K-Nearest Neighbors (K=7)
0	Akurasi (%)	93.961302	95.890981
1	Presisi (%)	99.082569	90.017668
2	Recall (%)	31.839623	60.082547
3	F1-Score (%)	48.192771	72.065064

Cell 15: Simulasi Inferensi Prediksi Pasien Baru

Maksud dan Tujuan Cell: Menguji kemampuan operasional model final KNN yang sudah dilatih untuk melakukan inferensi prediksi langsung pada profil pasien baru yang belum pernah dilihat sebelumnya. Dibuat fungsi tes_prediksi_pasien() yang menerima kamus data klinis, melakukan penskalaan fitur secara otomatis, dan mengeluarkan vonis diagnosis serta tingkat keyakinan probabilitas model (confidence level).

Kode Program:

```
# Muhammad Hafizh Wijdan - 202431005

# fungsi buat tes prediksi data pasien baru
def tes_prediksi_pasien(profil_dict, nomor_pasien):
    # bikin dataframe 1 baris sesuai urutan kolom X
    df_p = pd.DataFrame([profil_dict])[X.columns]
    # lakukan scaling yang sama persis kayak data train
    df_p_scaled = scaler.transform(df_p)

    # tebak status pakai knn
    pred = knn_final.predict(df_p_scaled)[0]
    prob = knn_final.predict_proba(df_p_scaled)[0]

    status_str = "POSITIF DIABETES (1)" if pred == 1 else "NEGATIF DIABETES (0)"
    confidence = prob[pred]  100

    print(f"=== HASIL PREDIKSI PASIEN #{nomor_pasien} ===")
    print(f"Usia: {profil_dict['age']} th | BMI: {profil_dict['bmi']} | HbA1c:
{profil_dict['HbA1c_level']}% | Glukosa: {profil_dict['blood_glucose_level']} mg/dL")
    print(f">> Hasil Prediksi KNN : {status_str}")
    print(f">> Tingkat Keyakinan  : {confidence:.2f}%\n")

# data pasien 1: kondisi sehat normal
pasien_sehat = {
    'gender': 0, 'age': 25.0, 'hypertension': 0, 'heart_disease': 0,
    'bmi': 21.5, 'HbA1c_level': 4.8, 'blood_glucose_level': 95,
    'smoking_history_current': 0, 'smoking_history_ever': 0,
    'smoking_history_former': 0, 'smoking_history_never': 1, 'smoking_history_not
current': 0
}

# data pasien 2: lansia dengan gula darah dan hba1c tinggi
pasien_risiko = {
    'gender': 1, 'age': 65.0, 'hypertension': 1, 'heart_disease': 1,
    'bmi': 34.2, 'HbA1c_level': 7.9, 'blood_glucose_level': 220,
    'smoking_history_current': 1, 'smoking_history_ever': 0,
    'smoking_history_former': 0, 'smoking_history_never': 0, 'smoking_history_not
current': 0
}

# jalankan fungsi prediksi
tes_prediksi_pasien(pasien_sehat, 1)
tes_prediksi_pasien(pasien_risiko, 2)
```

Penjelasan Kode Per Baris:

Baris 1: Watermark identitas mahasiswa.

Baris 4: Membuat fungsi inferensi tes_prediksi_pasien yang menerima data dictionary pasien baru dan nomor urut.

Baris 6-8: Mengubah dictionary menjadi DataFrame satu baris dan melakukan penskalaan fitur dengan scaler data latih.

Baris 11-12: Menghitung prediksi status biner memakai predict() dan probabilitas keyakinan memakai predict_proba().

Baris 14-15: Mengonversi output angka menjadi teks status diagnosa dan menghitung persentase keyakinan model.

Baris 17-20: Mencetak ringkasan profil klinis pasien dan hasil diagnosis ke layar konsol.

Baris 23-36: Mendefinisikan profil data Pasien 1 (kondisi sehat normal) dan Pasien 2 (kondisi risiko tinggi komorbid).

Baris 39-40: Menjalankan fungsi prediksi untuk kedua pasien baru tersebut.

Hasil Output dan Maknanya:

Hasil Output dan Maknanya:

- Hasil Prediksi Pasien #1:

Profil: Usia 25 tahun, BMI 21.5, HbA1c 4.8%, Glukosa 95 mg/dL (gaya hidup sehat, tanpa riwayat hipertensi/jantung)

Vonis Model: NEGATIF DIABETES (0)

Tingkat Keyakinan: 100.00%

- Hasil Prediksi Pasien #2:

Profil: Usia 65 tahun, BMI 34.2 (obesitas), HbA1c 7.9% (tinggi), Glukosa 220 mg/dL (hiperglikemia), hipertensi dan perokok

Vonis Model: POSITIF DIABETES (1)

Tingkat Keyakinan: 85.71%

- Makna Inferensi: Membuktikan bahwa model KNN yang dibangun berhasil menangkap pola korelasi medis di dunia nyata dengan sangat logis dan akurat (mampu membedakan dengan tegas pasien sehat berkeyakinan 100% dan pasien berisiko tinggi diabetes dengan keyakinan 85,7%).

[15]:
```
# Muhammad Hafizh Wijdan - 202431005

# fungsi buat tes prediksi data pasien baru
def tes_prediksi_pasien(profil_dict, nomor_pasien):
    # bikin dataframe 1 baris sesuai urutan kolom X
    df_p = pd.DataFrame([profil_dict])[X.columns]
    # lakukan scaling yang sama persis kayak data train
    df_p_scaled = scaler.transform(df_p)

    # tebak status pakai knn
    pred = knn_final.predict(df_p_scaled)[0]
    prob = knn_final.predict_proba(df_p_scaled)[0]

    status_str = "POSITIF DIABETES (1)" if pred == 1 else "NEGATIF DIABETES (0)"
    confidence = prob[pred] * 100

    print(f"=== HASIL PREDIKSI PASIEN #{nomor_pasien} ===")
    print(f"Usia: {profil_dict['age']} th | BMI: {profil_dict['bmi']} | HbA1c: {profil_dict['HbA1c_level']}% | Glukosa: {profil_dict['blood_glucose_lev
    print(f">> Hasil Prediksi KNN : {status_str}")
    print(f">> Tingkat Keyakinan  : {confidence:.2f}%\n")

# data pasien 1: kondisi sehat normal
pasien_sehat = {
    'gender': 0, 'age': 25.0, 'hypertension': 0, 'heart_disease': 0,
    'bmi': 21.5, 'HbA1c_level': 4.8, 'blood_glucose_level': 95,
    'smoking_history_current': 0, 'smoking_history_ever': 0,
    'smoking_history_former': 0, 'smoking_history_never': 1, 'smoking_history_not current': 0
}

# data pasien 2: lansia dengan gula darah dan hba1c tinggi
pasien_risiko = {
    'gender': 1, 'age': 65.0, 'hypertension': 1, 'heart_disease': 1,
    'bmi': 34.2, 'HbA1c_level': 7.9, 'blood_glucose_level': 220,
    'smoking_history_current': 1, 'smoking_history_ever': 0,
    'smoking_history_former': 0, 'smoking_history_never': 0, 'smoking_history_not current': 0
}

# jalankan fungsi prediksi
tes_prediksi_pasien(pasien_sehat, 1)
tes_prediksi_pasien(pasien_risiko, 2)
```

202431005_Muhammad Hafizh Wijdan

```
=== HASIL PREDIKSI PASIEN #1 ===
Usia: 25.0 th | BMI: 21.5 | HbA1c: 4.8% | Glukosa: 95 mg/dL
>> Hasil Prediksi KNN : NEGATIF DIABETES (0)
>> Tingkat Keyakinan  : 100.00%

=== HASIL PREDIKSI PASIEN #2 ===
Usia: 65.0 th | BMI: 34.2 | HbA1c: 7.9% | Glukosa: 220 mg/dL
>> Hasil Prediksi KNN : POSITIF DIABETES (1)
>> Tingkat Keyakinan  : 85.71%
```

BAB IV
PENUTUP

Berdasarkan landasan teori serta rangkaian eksperimen pengolahan data dan pemodelan Machine Learning yang telah dilakukan pada dataset rekam medis 02_diabetes.csv, dapat ditarik kesimpulan menyeluruh yang merangkum keterkaitan antara teori dan hasil praktikum. Secara teoritis dan praktis, alur kerja Supervised Learning terbukti sangat bergantung pada kualitas tahapan Exploratory Data Analysis (EDA) dan Data Preprocessing sebelum data dimasukkan ke dalam algoritma pemodelan. Dari total 100.000 baris dan 9 kolom data awal, proses EDA berhasil mengidentifikasi adanya ketimpangan kelas (class imbalance) yang cukup besar antara pasien Non-Diabetes sebesar 91,5% (91.500 orang) dan pasien Diabetes sebesar 8,5% (8.500 orang), serta membuktikan bahwa indikator biokimia darah yaitu blood_glucose_level ($r = +0{,}42$) dan HbA1c_level ($r = +0{,}40$) memiliki korelasi positif paling kuat terhadap status diabetes dibandingkan faktor usia ($r = +0{,}26$) maupun BMI ($r = +0{,}21$).

Tahapan preprocessing yang meliputi pembersihan 3.854 baris data duplikat (3,85%) dan 18 baris anomali kategori gender 'Other' berhasil menghasilkan 96.128 baris data bersih yang bebas dari risiko kebocoran data (data leakage) antara training set (76.902 baris) dan testing set (19.226 baris). Transformasi atribut teks menjadi angka melalui Label Encoding dan One-Hot Encoding (menghasilkan 12 kolom fitur prediktor), yang kemudian diikuti dengan penyamaan skala menggunakan StandardScaler, terbukti krusial bagi algoritma berbasis perhitungan jarak (distance-based) seperti K-Nearest Neighbor (KNN). Tanpa standarisasi skala, atribut dengan rentang nilai ratusan seperti kadar glukosa darah (80–300 mg/dL) akan mendominasi rumus Euclidean Distance dan mengabaikan kontribusi fitur berskala kecil seperti kadar HbA1c (3,5–9,0%). Pada pengujian pemodelan kasus **regresi nilai kontinu** untuk mengestimasi kadar glukosa darah (blood_glucose_level) berdasarkan profil klinis lainnya, baik algoritma Linear Regression ($R^2 = 0{,}0501$, MAE $= 30{,}95$ mg/dL, RMSE $= 39{,}99$ mg/dL) maupun KNN Regressor dengan $K = 7$ ($R^2 = -0{,}0414$, MAE $= 33{,}05$ mg/dL, RMSE $= 41{,}87$ mg/dL) menunjukkan bahwa fluktuasi angka glukosa darah harian seseorang bersifat kompleks dan sulit ditebak secara presisi hanya dari rata-rata kemiripan profil demografis atau persamaan garis lurus sederhana, meskipun pendekatan parametrik Linear Regression sedikit lebih stabil dibandingkan rata-rata lokal KNN Regressor. Sebaliknya, pada pemodelan penentuan **status diagnosis diabetes**, algoritma non-parametrik K-Nearest Neighbor (KNN) menunjukkan performa yang jauh melampaui pendekatan Linear Probability Model (Regresi Linier dengan threshold 0,5). Melalui proses hyperparameter tuning pada rentang nilai K ganjil (1 hingga 15), ditemukan bahwa $K = 1$ mengalami overfitting ekstrem (akurasi latih 99,99% namun akurasi uji turun ke 93,60%), sedangkan konfigurasi optimal tercapai pada rentang $K = 5$ hingga $K = 7$ yang memiliki keseimbangan generalisasi paling stabil. Penerapan model final KNN ($K = 7$) pada seluruh data uji menghasilkan akurasi akhir sebesar **95,89%**, unggul dibandingkan Regresi Linier yang memperoleh akurasi 93,96%. Keunggulan paling krusial terlihat pada metrik Recall (sensitivitas deteksi pasien positif diabetes) dan F1-Score: Regresi Linier hanya mampu mencapai Recall **31,84%** dan F1-Score **48,19%**, sedangkan KNN ($K = 7$) berhasil meningkatkan Recall hampir dua kali lipat menjadi **60,08%** (1.019 True Positive dan 17.417 True Negative) dengan F1-Score sebesar **72,07%**. Simulasi inferensi pada data pasien baru juga membuktikan bahwa model KNN yang telah dilatih mampu membedakan profil pasien sehat (keyakinan 100,00%) dan pasien lansia berisiko tinggi (keyakinan 85,71%) secara logis dan akurat sesuai kaidah medis.

DAFTAR PUSTAKA

[1] Lestari, & Faisal, S. (2024). Implementation of Diabetes Prediction Model Using Random Forest Algorithm, K-Nearest Neighbor, and Logistic Regression. *Jurnal Teknik Informatika (JUTIF)*, 5(4), 1165–1174. https://jutif.if.unsoed.ac.id/index.php/jurnal/article/view/2593

[2] Montes-Bustamante, J. (2023). Diabetes Predictive Model Using the CRISP-DM Process for Disease Prevention Implementing Machine Learning. *Ingeniería e Innovación*, 4(2), 17–37.

[3] Moradi, M., et al. (2025). Boosting K-nearest neighbor regression performance for longitudinal data. *BMC Medical Research Methodology / PubMed Central (PMC)*. https://pmc.ncbi.nlm.nih.gov/articles/PMC12482645/

[4] Srisuradetchai, P., & Suksrikran, K. (2024). Random kernel k-nearest neighbors regression. *Frontiers in Big Data*, 7, 1402384. https://doi.org/10.3389/fdata.2024.1402384

[5] Srinivasu, P. N., Shafi, J., Krishna, T. B., Sujatha, C. N., Praveen, S. P., & Ijaz, M. F. (2022). Diabetes mellitus prediction and diagnosis from a data preprocessing and machine learning perspective. *Computer Methods and Programs in Biomedicine*, 220, 106773. https://doi.org/10.1016/j.cmpb.2022.106773

[6] Pagan, M., Zarlis, M., & Candra, D. (2023). Investigating the impact of data scaling on the k-nearest neighbor algorithm. *Computer Science and Information Technologies (CSIT)*, 4(2), 135–142. https://doi.org/10.11591/csit.v4i2.pp135-142

[7] Ozcan, A., et al. (2025). Comprehensive blood glucose level prediction from HbA1c using machine learning framework. *Scientific Reports (Nature Portfolio)*, 15, 20234. https://doi.org/10.1038/s41598-025-20234-z

[8] Kumar, A., et al. (2025). K-Nearest Neighbors (KNN) Regression as a Tool for Failure Rate Prediction. *Defence Science Journal*, 75(6), 771–778.

[9] Kurniawan, M. F., & Megawaty, D. A. (2025). Comparison of Logistic Regression, Random Forest, Support Vector Machine (SVM) and K-Nearest Neighbor (KNN) Algorithms in Diabetes Prediction. *Journal of Applied Informatics and Computing (JAIC)*, 9(5), 2154–2162. https://jurnal.polibatam.ac.id/index.php/JAIC/article/view/9815

[10] Pratama, R., et al. (2025). Comparative Analysis of Machine Learning Regression Algorithms and Evaluation Metrics (R-Squared, RMSE, MAE). *International Journal of Research and Applied Technology (INJURATECH)*. https://ojs.unikom.ac.id/index.php/injuratech/article/view/20743